

TU Dublin - City Campus Online Dissertation Copyright Policy

The TU Dublin City Campus Library Online Dissertation Collection is comprised of undergraduate and postgraduate taught dissertations.

The copyright of a dissertation resides with the author. All TU Dublin students are bound to comply with Irish copyright legislation during their course of study. A dissertation is made available for consultation on the understanding that the reader will not publish it in any form, either the whole or any other part of it, without the written permission of the author.

To learn more about copyright and how it applies to you click [here](#):

The Online Dissertation Collection is only available to be accessed by registered TU Dublin City Campus staff and students. Non-members of TU Dublin cannot be given access to the Online Dissertation Collection. (Please note that this also applies to graduates of TU Dublin and graduates of the former Dublin Institute of Technology requiring a copy of their own work). In using the Online Dissertation Collection students should be fully aware of the following points:

- Usernames and passwords are for personal use only. They must not be divulged for use by any other person.
- The Online Dissertation Collection is provided for personal educational purposes to support you in your courses of study. Use of this collection does not extend to any non-educational or commercial purpose.
- You must not email copies of any dissertation or print out copies for anyone else. You must not under any circumstances post content on bulletin boards, discussion groups or intranets etc.
- You must not remove, obscure or alter in any way any copyright information or watermarks that appear on any material that you download/print from the collection.

TU Dublin City Campus Library has to right to temporarily or permanently remove a digitised dissertation from the collection.

Artificial Intelligent Voice Assistant Final Project Report

**DT228
BSc in Computer Science**

Angesh Kumar Chanderdip

D20125301

Marisa Llorens Salvador

School of Computer Science
Technological University, Dublin

Date: 27th March 2023

Abstract

Intelligent Voice assistants are accessible in our daily lives through our smart devices. Microsoft's Cortana, Apple's Siri, Google Assistant, Samsung's Bixby are all supporting us as we get through the day with various tasks, such as setting reminders or calling a person with just a simple voice command. Furthermore, voice assistants have extended to households' appliance with smart speakers like Amazon's Alexa or Google Home to provide voice over control on household appliances. Nowadays, Voice assistant has become very widely used. However, there is still a limited focus in providing a better emphasis for disabled user in the market. This project is being undertaken to shape a user-friendly voice assistant for desktop devices to promote independency within the disabled community. Thus, this will be an inclusion tool that will help to empower disabled people by making them feel less reliable on others and become more confident.

According to a new report by the World Health Organisation, almost one billion children and adults with disabilities, as well as older persons, do not have access to the assistive technology they need, and WHO's chief urged all the countries to make this issue a priority. (Peseckyte, 2022). As we progress and advance in a technological world, there is a gap somewhere along the way for user with disabilities.

VOYA is an intelligent voice assistant that operates with voice recognition, language processing algorithms, and voice synthesis to listen to specific voice commands and returns real-time information or performs specific functions as requested by the user.

Declaration

I hereby declare that the work described in this dissertation is, except where otherwise stated, entirely my own work and has not been submitted as an exercise for a degree at this or any other university.

Signed:

Angesh Kumar Chanderdip

Angesh Kumar Chanderdip

27 March 2023

Acknowledgements

Thank you to my supervisor, Marisa Llorens Salvador, for providing guidance and feedback throughout this project. Thanks also to all the hardworking staff of the Technological University Dublin.

A special thank you to my girlfriend Deeya, who has provided valuable insights and guidance in the approach of this project.

Research / Private Study Only

Table of Contents

1. Introduction.....	11
1.1. Project Background	11
1.2. Project Description	12
1.3. Project Aims and Objectives	12
1.4. Challenges: NLP algorithm	13
1.5. Project Scope.....	14
1.6. Thesis Roadmap	14
1.6.1 Literature Review.....	14
1.6.2 System Design.....	14
1.6.3 Prototype Development.....	14
1.6.4 Software Development	14
1.6.5 Testing and Evaluation	14
1.6.6 Issues and Future Work	14
2. Literature Review.....	15
2.1. Introduction	15
2.2. Alternative Existing Solutions to Your Problem	15
2.2.1 Apple's SIRI	16
2.2.2 Voiceitt	17
2.2.3 Amazon's "Alexa"	17
2.3. NLP for voice input understanding.....	18
2.3.1 Natural Language Processing	19
2.3.2 How NLP works.....	19
2.3.3 NLP as a service in Application	20
2.3.4 Large Language Models Researched	20
2.4 Neural Networks and Deep Learning	24
2.4.1 Intent Classification	25
2.4.2 Types of Deep Learning algorithm.....	27
2.4.3 Datasets for training model.....	29
2.4.4 Chosen Deep Learning algorithm and dataset.....	30
2.5. Programming Languages researched	31
2.5.1 JAVA	31
2.5.2 Lisp	31

2.5.3 Python	31
2.5.4 Chosen Programming Language	32
2.6. Python libraries Researched	32
2.6.1 Speech Detection	32
2.6.2 Convert text to speech	33
2.6.3 Graphical User Interface	34
2.6.4 Deep Learning	35
2.6.5 Application Programming Interfaces - APIs	36
2.6.6 Chosen libraries	36
2.7. Hardware Requirements	37
2.8. Existing Final Year Projects	37
2.8.1 Title: NLPurchase – eCommerce Chatbot	37
2.8.2 Title: AAC Using Natural Language Processing	37
2.9. Conclusion	38
3. Software Design	38
3.1. Introduction	38
3.2. Software Design	38
3.2.1. High Level Fidelity prototype	39
3.2.2 Use case diagram	41
3.2.3 Class diagram	42
3.2.4 Sequence diagram	43
3.2.5 Activity diagram	43
3.2.6 Software Methodology	45
3.2.7 GANTT Chart	49
3.3. Features of the System	50
3.3.1 Task-oriented approach	50
3.3.2 Knowledge-oriented approach	51
3.4. Software Test Plan	51
3.4.1 CONTINUOUS TESTING (MONITORING)	52
3.4.2 MANUAL-BASED TESTING	52
3.4.3 USABILITY PERFORMANCE TESTING	52
3.4.4 UNIT TESTING	52
3.4.5 Training Model Evaluation	52

3.5. Architecture for voice assistant	53
3.5.1 Front end	53
3.5.2 Middle Tier	53
3.5.3 Backend	55
3.6. Analysis of a command	56
3.6.1 Wake word	56
3.6.2 Request.....	56
3.6.3 Expression.....	56
3.7. Conclusion.....	56
4. Software Development.....	57
4.1. Introduction	57
4.2 Software Development.....	57
4.3. Front End.....	58
4.3.1 Model-View-Controller	58
4.3.2 Code snippet.....	59
4.4 Middle Tier	62
4.4.1 Speech Recognition.....	62
4.4.2 Intent Classification	64
4.5 Backend	67
4.5.1 API-Based	67
4.5.2 Author based function	70
4.6 Challenges and problems	74
4.6.1 Wake up word	74
4.6.2 Training Model.....	75
4.6.3 Voice User Interface.....	75
4.6.4 GDPR Law for voice data	75
4.6.5 Accents	76
4.7 Conclusion	76
5. Evaluation.....	77
5.1 Introduction	77
5.2 Software Evaluation	77
5.2.1 Evaluation Criteria	77
5.2.2 Plan for evaluation.....	78

5.2.3 CONTINUOUS TESTING (MONITORING).....	78
5.2.4 UNIT TESTING	79
5.3 Specific evaluation.....	80
5.3.1 Training Model Evaluation	80
5.4 Questionnaires and Interviews Evaluation.....	81
5.4.1 Questionnaire	81
5.4.2 Interviews	82
5.4.3 Positive feedback	83
5.4.4 Constructive criticisms	83
5.5 Conclusion.....	83
6. Conclusion and Future Plan.....	84
6.1 Introduction	84
6.2 Conclusion.....	84
6.3 Future Plan.....	85
6.3.1 Database integration	85
6.3.2 Machine learning for predictive word.....	85
Bibliography	86
Appendix:	91

Table of Figures

Figure 1: Image of how a voice assistant work with NLP	13
Figure 2: Screenshot of SIRI from Apple	16
Figure 3: Screenshot of Voiceitt	17
Figure 4: Image of Amazon's Alexa interface.....	18
Figure 5: How ChatGPT works.....	21
Figure 6: An example of ChatGPT's interface	22
Figure 7: BLOOM Architecture	23
Figure 9: Overview of Neurons in machine learning	24
Figure 10: Formulae to calculate the inputs	24
Figure 11: Formulae to calculate all the input and bias	25
Figure 12: Formulae to calculate the activation function	25
Figure 13: Microsoft In-app view. Initial state when voice activation begins.	39
Figure 17: High-level fidelity prototype of VOYA	40
Figure 18: High-level fidelity prototype of VOYA	40
Figure 19: High-level fidelity prototype of VOYA	41
Figure 20: Use case diagram of Opening Word document.....	41
Figure 21: Use case diagram of Getting date and time	42
Figure 22: Class diagram of VOYA	42
Figure 23: Sequence diagram for Task Oriented Approach.....	43
Figure 24: Sequence diagram for Task Oriented Approach.....	43
Figure 25: Activity diagram of VOYA executing a command	44
Figure 54:Gantt Chart	50
Figure 14: Testing's stages	51
Figure 15: Architecture of VOYA	55
Figure 16: Overview of command for VOYA	56
Figure 26: Model View Controller Architecture.....	58
Figure 27: Snippet for initializing Voice user Interface.....	59
Figure 28: Snippet for display output with threading	60
Figure 29: Snippet for Voice user Interface-Welcome	61
Figure 30: Snippet for Voice user Interface – Keyword detection	62
Figure 31: Snippet for initializing the Voice assistant object.....	62
Figure 32: Snippet of the listen() method.....	63
Figure 33: Snippet of the Wake word detection.....	63
Figure 34: Snippet of the text to speech method	64
Figure 35: Snippet of dataset download.....	64
Figure 36: Snippet of concatenating both intents.....	65
Figure 37: Snippet of fitting the model.....	66
Figure 38: Snippet of model training	67
Figure 39: Snippet of retrieving weather data.....	67
Figure 40: Snippet of the weather information in the JSON	68
Figure 41: Snippet of ChatGPT method initialisation	68
Figure 42: Snippet of ChatGPT model	69

Figure 43: Snippet of the dream studio model	70
Figure 44: Snippet of generating the image from dream studio	70
Figure 45: Snippet of WhatsApp text message function	71
Figure 46: Snippet of email function	72
Figure 47: Snippet of the date function.....	72
Figure 48: Snippet of the time function.....	73
Figure 49: Snippet of the system stat info to return back.....	73
Figure 50: Snippet of converting the stat into human readable format	74
Figure 51: Snippet for evaluating the training model	80
Figure 52: Test set Accuracy metrics	81
Figure 53: Train set Accuracy metrics.....	81

Research / Private Study Only

1. Introduction

In this evolving world, artificial intelligence and technological advancements are altering life rapidly. Artificial intelligence is being used more than ever in both our personal and professional life. Change is not always easy and mostly when it comes to more vulnerable people, such as people who live with certain disabilities, it is harder for them to adapt. An intelligent voice assistant can provide more autonomy and independency to disabled users. The main objective of this project is to take into consideration the rapid pace of technological change and help people with disabilities to use technology independently.

This section introduces the concepts of how an intelligent voice assistant can provide more autonomy and independency to a disabled user. This section also addresses the objectives and challenges of this project.

1.1. Project Background

VOYA is a virtual intelligent assistant that help user with disability to perform computer related task without the need to physically interact with the hardware.

As we grow in awareness, public companies, governments, and private enterprises are looking for ways to be more inclusive through improving accessibility for people with different physical abilities. Most of the application we use in our daily life are not adapted towards the disabled community. Generally, the implementation is harder to optimize for a website, mobile app, or tablet to provide the same accessibility for people with a wide variety of needs. For example, a percentage of user require specialized input of text, and others need communication without relying on a screen.

According to DFI, 643,131 people (13.5%) in the Republic of Ireland, has a form of disability and according to the CDC, 61 million adults (26%) in the United States live with a disability. (DFI Infographic IRELAND, 2019) Making products more accessible for this underserved and largely untapped market is not only good business, but also the right approach toward helping people. One solution is to voice-enable products and services to eliminate the need to input text, navigate a series of types and swipes, or access a screen. (Disability and Health Promotion, 2022)

The inspiration of this project lies in the author's background as a health care provider where each day he encountered people with a physical disability unable to enjoy the perks of what technology has to offer. As his day goes by and all the other patients were entertaining themselves with technology in the form of social media or entertainment, there is a percentage that were unable to do so without needing help from a secondary

person to perform the manual tasks. There are many benefits in the utilisation of technology such as staying in touch with your loved one, entertainment, education, self-development which will provide a well-balanced mental state as well as provide the sense of independency. However, these tasks proved to be difficult for the disabled community.

VOYA is trying to minimise this disparity by providing the digital support for users with their daily technological activities and open the door to independency. VOYA will focus more on features towards helping disabled people such as medication reminder, as according to a research, disabled people receive an average of 40% more prescription medication annually. (Residents, 2000)

1.2. Project Description

VOYA is an intelligent voice assistant that operates with voice recognition, language processing algorithms, and voice synthesis to listen to specific voice commands and returns real-time information or performs specific functions as requested by the user. The user can activate the personal assistant by saying the keyword "Hey VOYA". VOYA will wait for a command by voice input or text mode to perform an action. For people with physical impairments, the voice input will be practical since it would be easier to say a few words rather than typing them. Voya will take into consideration different categories of users' disabilities such as visual impairment, motor and cognitive disabilities and try to solve that gap with technology. For example, Voya can easily be a digital personal assistant to a blind user by using voice input to perform action within their laptop. (Borycka, 2021)

VOYA will include the following features: chat services through WhatsApp or messenger, text messaging, mail composition, setting alarm, event handler, location services, music player service, checking weather, Google searching engine, Wikipedia searching engine, maintain conversation, tell jokes and so more. Voya will use a device's microphone to receive voice requests while the voice output takes place at the speaker. It is a combination of several different technologies: voice recognition, natural processing language, voice analysis and the transition from speech to text.

1.3. Project Aims and Objectives

The overall aim of this project is to build an intelligent voice assistant mainly focussed on providing independency for user with disability by minimising manual computer related tasks. This will be achieved by developing a graphical voice user interface (VUI) as the application's frontend such that it will provide a dialog box for the user. Once the project is completed, the application will have the following functionalities:

- i. Perform simple knowledge task as the user provide a voice input command,
- ii. Able to operate opening and closing an application,

- iii. Ability use chat services such as WhatsApp and messenger,
- iv. Able to compose a quick email,
- v. Set alarm and reminders,
- vi. Carry a Google search or Wikipedia by asking question,
- vii. Maintain a normal conversation,
- viii. Able to Tell jokes.

1.4.Challenges: NLP algorithm

The biggest challenge of implementing the voice assistant is the understanding of human language. Computers cannot naturally understand human speech. Instead, programming languages are used to provide a set of instructions on how to perform. In order for VOYA to understand command, Natural language processing is required to provide the ability to bridge the gap of communication between human and computers. NLP is a combination of computational semantics with statistical, machine learning, and deep learning models. Collectively, these technologies enable computers to process human language in the form of voice data (Kuchi, 2016) as shown in figure 1, (Kellner, 2020).



Figure 1: Image of how a voice assistant work with NLP [22]

The aim of the NLP algorithm will be on understanding the intent of the user's command. The benefit of natural language processing can be demonstrated when analysing the following two statements: "Hey voya, search for nearest resto" and, "Hey voya, recommend a restaurant to eat". Natural language processing will recognize that restaurant is an entity, and that resto is an abbreviated form of restaurant.

1.5. Project Scope

This project is being devised to shape a user-friendly voice assistant for desktop devices to promote independency within the disabled community. VOYA is an intelligent voice assistant that operates with voice recognition, language processing algorithms, and voice synthesis to listen to specific voice commands and returns real-time information or performs specific functions as requested by the user. The user can activate the personal assistant by saying the keyword “Hey VOYA”. VOYA will wait for a command by voice input or text mode to perform an action. This is ideal as it is easier to say a few words than typing them. Voya will take into consideration different categories of users’ disabilities such as visual impairment, motor and cognitive disabilities and try to solve that gap with technology. For example, Voya can easily be a digital personal assistant to a blind user by using voice input to perform action within their laptop. (Borycka, 2021)

1.6. Thesis Roadmap

This section will provide an overview of what the following chapters are about.

1.6.1 Literature Review

The objective of this Literature Review is to explore existing solutions to the problem that VOYA attempts to overcome. It also explores how functions of these existing solutions can be incorporated into VOYA using the researched technologies.

1.6.2 System Design

This chapter outlines the development approach to be taken during the project. It accomplishes this by explaining the chosen methodology. And also include an approach to the system overview.

1.6.3 Prototype Development

This section provides a detailed overview of the development process to include a high-level fidelity prototyping, use case diagram, class diagram, sequence diagram and activity diagram.

1.6.4 Software Development

This section provides a detailed overview of the development process to include the implementation of the front-end, middle tier and back-end.

1.6.5 Testing and Evaluation

The aim of this chapter is to define the plan for the application’s testing and evaluation going forward.

1.6.6 Issues and Future Work

The goal of this section is to evaluate the issues faced during the development of the application or possible future issues that might occur. And a discussion on the future implementation to be carried out on the project.

2. Literature Review

2.1. Introduction

This chapter is a literature review on the topics of intelligent voice assistant to bridge the gap between disability and technology. The aim and objectives are to provide disabled users with a more convenient lifestyle with accessible technology. VOYA is an intelligent voice assistant that operates with voice recognition, language processing algorithms, and voice synthesis to listen to specific voice commands and returns real-time information or performs specific functions as requested by the user.

In this chapter, the focus will be towards the different solution and technologies used to overcome the gap between technologies and disabled user. This project is focused on helping users with disabilities to get a more convenient lifestyle and independency with technology and helps users to minimise for manual operations with their laptop or PC.

Additionally, this section will be discussed on the existing solution in the market right now and understand the approach these companies took to achieve a solution. Apple's Siri, Voiceitt and Amazon's Alexa are the three main software this research is conducted on as there are many similarities with VOYA in terms of text to speech technologies, Natural language processing and interface. SIRI provide a great understanding to assistive technologies while Voiceitt provide an understanding to the speech recognition with disabled user and how to overcome these obstacles.

Research carried on the technologies to achieve this project are Python, JAVA, Lisp which are all AI supported programming language. They all provide a voluminous library to support module needed to build the voice assistant. As well as research on how deep learning can be implemented to provide a much better understand of user's intent.

Furthermore, research was carried out on similar project from the college library website. Two projects, namely NLPurchase – eCommerce Chatbot and AAC Using Natural Language Processing were conducted with similar ideas, focus and technology as for this project. Both projects implement a natural language interface and provide a focus to AI supported interfaces.

2.2. Alternative Existing Solutions to Your Problem

This section will cover on the research conducted on similar solution available in the market in bridging the gap between technologies and disabled user. The improvement of voice assistant is ongoing, currently there are few that are adapted for disabled users. However,

they do not address the need for every user. For the purpose of this section, we will discuss on Apple's Siri, Voiceitt and Amazon's Alexa.

2.2.1 Apple's SIRI

SIRI is a voice assistant with a well-developed assistive technology that is available on the macOS devices and IOS devices only. SIRI make use of voice requests with a natural-language user interface to answer questions and perform actions by delegating requests to a set of internet services. As a personal assistant, Siri accesses contacts and calendar data to support driven requests. Additionally, Siri requires to be connected to the internet.

The primary language of apple software is Objective-C as most of Apple's code base is in Objective-C although Apple is migrating to Swift at a very fast speed. Siri consists of many modules like AI engine, Natural Language Processing, Machine Learning, Data Transfer and Optimization. (sirikit, 2022)

For a worldwide known software like SIRI, it has one security issue. To be specific, if an iPhone is password protected, Siri can be invoked with the Home button, which can be used to send messages and make calls to people in contacts list without unlocking the phone. Siri has another flaw for which links were disruptive when user clicked on it in the result list return from SIRI, it were taken to the browser or to a different app, and user was not sure how to get back to the list to continue inspecting other results. (Budiu, 2018)

This project can benefit the minimalistic design from SIRI. A small window interaction occurs every time Siri responds with a feel of a very subtle vibration. It is essential for the intelligent assistant to have an excellent usability in respect to heuristics.

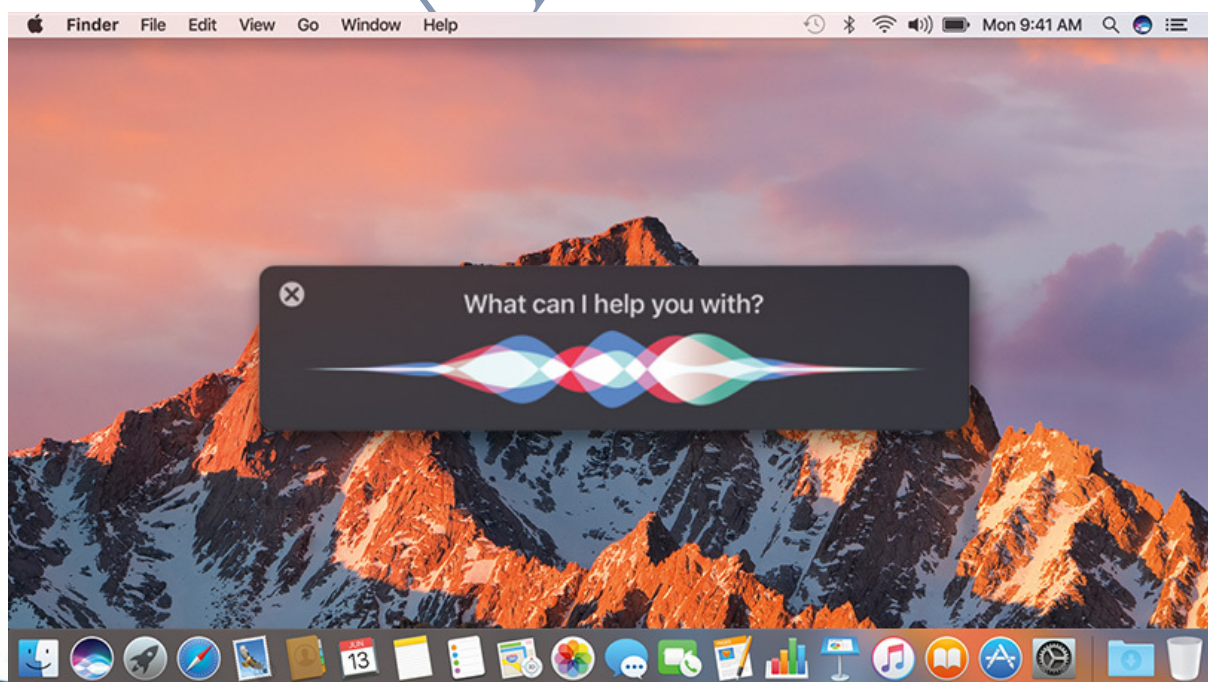


Figure 2: Screenshot of SIRI from Apple

2.2.2 Voiceitt

Voiceitt is a speech recognition app for people with speech disabilities, disorders, or impairments that uses machine learning to identify and learn unique speech. Voiceitt uses speech training to learn and model speech patterns to predict the desired output. It also uses different algorithm to learn unique speech patterns through statistical modelling and machine learning. This help user with speech disorder such as stuttering, apraxia, and dysarthria. Voiceitt is only available with iPhones and iPads that run iOS12 or above. And also, it can be used with voice activated devices like Alexa. (Voiceitt learns your clients' unique speech patterns, 2023)

The application allows the user to create a personalized dictionary with phrases of choice which in turn is used to train to recognize unique pronunciation of each phrase. (Melissa Moyer, 2020)

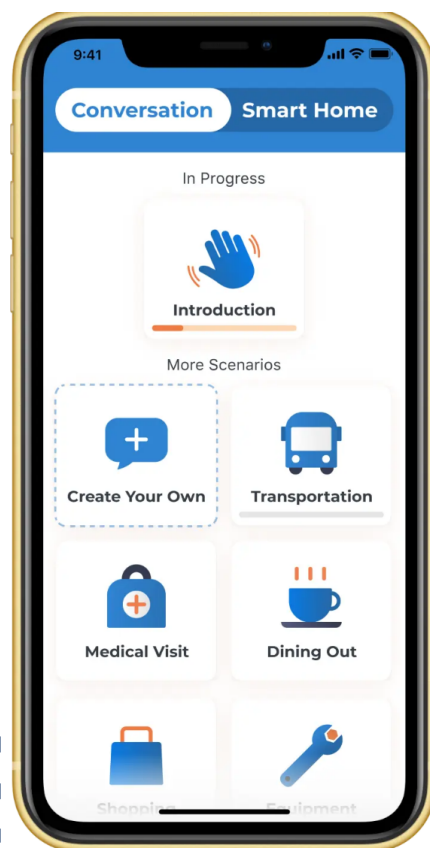


Figure 3: Screenshot of Voiceitt [51]

2.2.3 Amazon's "Alexa"

Alexa is a virtual assistant using Artificial Intelligence (AI) that was developed and produced by Amazon in 2015. The voice-activated personal assistant Alexa can be used to create lists and reminders, maintain appointment calendars, place a phone call or order from Amazon, provide news and weather updates, play music and answer queries. (Brown, 2018)

Through the interconnectivity with Smart Devices, users have the ability to create a Smart Home through the AI feature of Amazon Echo and Alexa to control a range of household devices such as sensors, smart bulb, security systems with a simple voice command. These functions provide independence for people with disabilities. (Irene Lopatovska, 2018)

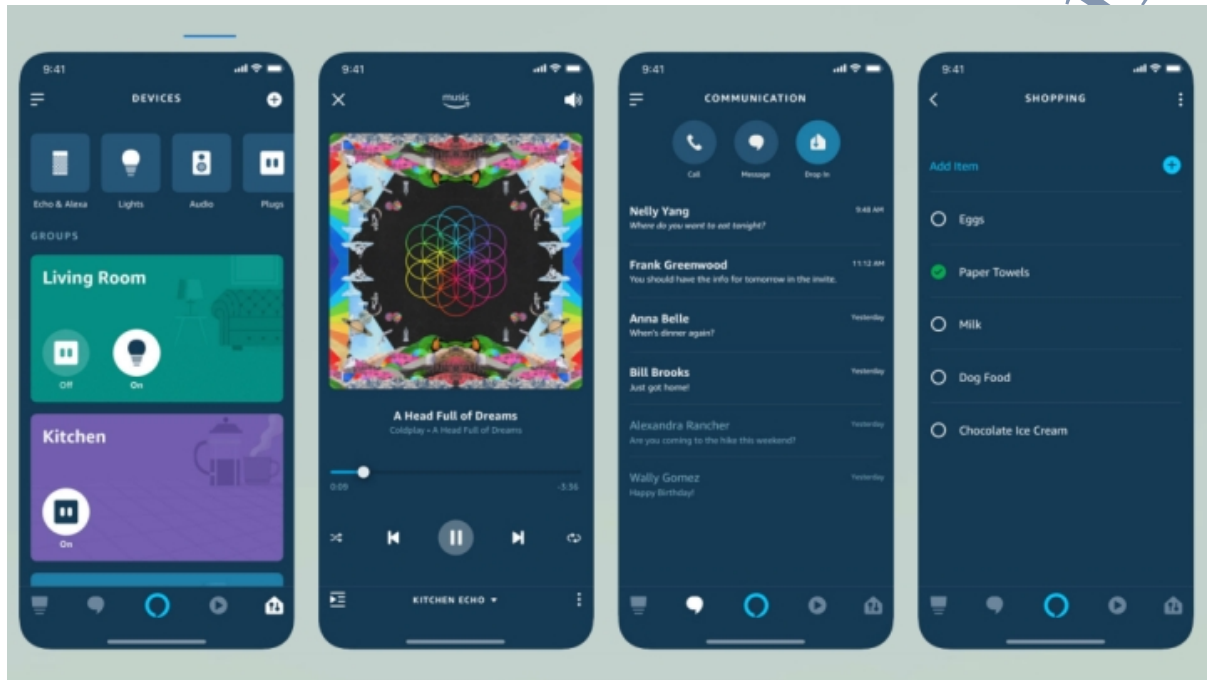


Figure 4: Image of Amazon's Alexa interface [20]

2.3. NLP for voice input understanding

Natural Language Processing (NLP) provides a mean for computers to understand and process human languages. The most popular application that implement NLP are virtual assistants, like Siri, Alexa, and Google Assistant. Natural language understanding is deemed to be a complex implementation in the area of artificial intelligence. The complexity of this computational problem is the ability for computers to be as intelligent as people and to understand human in complete. NLP will help VOYA automatically understand and analyse a variety of unstructured text data, for when receiving voice input from the user.

The primary complexity of this project is to build the NLP algorithm to make VOYA understand the meaning of the command in order to provide the correct output. Stemming is the fundamental of voice assistant to comprehend human languages.

The recent years came with a breakthrough in artificial intelligence (AI) with the introduction of language processing technologies. These Technologies have enabled a more intelligent systems with a richer understanding of language. Large pre-trained Transformer language models, also known as the large language models, have increases the abilities of what system can accomplish with words. The Language Models are the essence of cutting-

edge Natural Language Processing (NLP). In most NLP tasks, the Language Models can predict the next word by analysing the given text.

2.3.1 Natural Language Processing

Neuro-Linguistic Programming (NLP) take an instance with ELIZA as the first NLP program, a chatbot program which was developed from 1964 to 1966 (Ina, 2022) at the Artificial Intelligence Laboratory of MIT by Joseph Weizenbaum. It was a program which was based on script named as DOCTOR, which was arranged to Rogerian Psychotherapist and used rules, to response the questions of the users which were psychometric based. It was one of the chatbots which were capable of taking the Turing test at that time.

ELIZA uses "pattern matching" and substitution methodology, which in return the program gives real like responses that made early users feel they were talking to someone who understood their input. Although the system had no actual understanding of the thoughts or feelings being conveyed, many users believed that ELIZA was guided by true machine intelligence. Regarded as an academic curiosity for many years, NLP is finally making its way into practical applications. (Ina, 2022)

2.3.2 How NLP works

Artificial neural networks attempt to model the way human brain cells operate, albeit in a limited scope. There are many types of artificial neural networks that differ according to how the "neurons" are organized and how signals flow among them.

Each neuron takes an input and manipulates it mathematically to generate an output that is passed on to one or more other neurons. The neural network designer defines the mathematical function associated with each neuron. The collective processing of the neurons converts a set of inputs (such as the pixels in an image) to the desired output (identifying an object in the image). (Knight, 2019)

Machine-learning techniques take artificial neural networks a step further, by tweaking the mathematical functions of the neurons automatically over time to improve the accuracy of its results. Machine-learning algorithms must be "trained" using large amounts of annotated data so that it can go through many cycles of processing, evaluating, and tweaking.

In the case of NLP, the input is a block of text (either written text or text converted from speech), and the output is some desired characteristic of the text, such as its purpose (search query, command, review, etc.) and meaning or tone (positive, negative, angry, biased).

2.3.3 NLP as a service in Application

NLP has a wide range of applications in voice assistants, including:

- i. Natural Language Understanding - NLP enables voice assistants to understand and interpret user requests, allowing them to provide relevant and accurate responses.
- ii. Natural Language Generation - NLP can be used to generate natural-sounding responses to user requests, improving the overall user experience.
- iii. Sentiment Analysis - NLP can be used to analyse the sentiment behind user requests, allowing voice assistants to tailor their responses accordingly.
- iv. Personalization - NLP can be used to personalize the voice assistant experience based on the user's preferences and previous interactions with the system.
- v. Language Translation - NLP can be used to translate user requests into different languages, making voice assistants more accessible to a global audience.

2.3.4 Large Language Models Researched

2.3.4.1 ChatGPT

As ChatGPT is the new breakthrough in large language model, thus fall in the scope of this project. The author has decided to leverage ChatGPT services as it is closely correlated to the voice assistant requirements. ChatGPT is fairly recent, therefore there are a limited software in the market who can leverage its services and provide a voice to the text-based software. ChatGPT is a large language model developed by OpenAI based on the GPT (Generative Pre-trained Transformer) architecture released in December 2022. GPT-3 (Generative Pretrained Transformer 3) and GPT-4 are state-of-the-art language processing AI models developed by OpenAI. They can produce text that resembles that of a human and have a wide range of uses, including language translation, language modelling, and creating text for chatbots and other applications. With 175 billion parameters, GPT-3 is one of the most sophisticated and substantial language processing AI models available today. (Hughes, 2023)

It cannot produce video, sound, or images like Dall-E 2, but instead has an in-depth understanding of the spoken and written word.

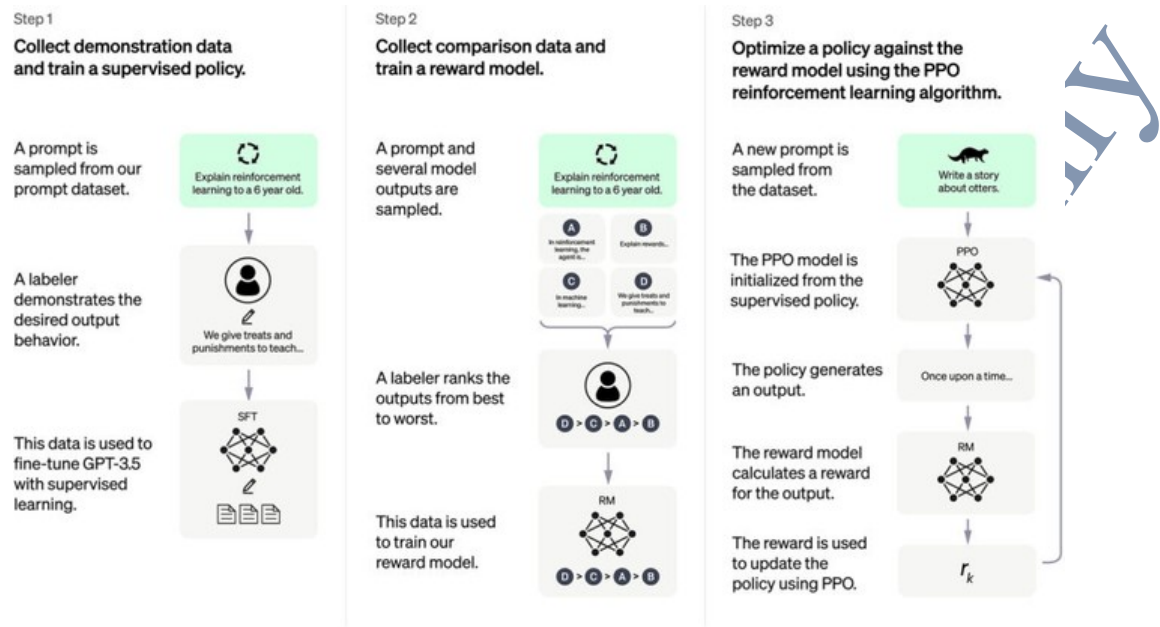


Figure 5: How ChatGPT works [18]

GPT-3 model was fine-tuned to create a supervised training dataset, where the input has a known output for the model to learn from. Prompts were collected from user entries into the Open API and the labellers wrote an appropriate response to create a known output. The GPT-3 model was then fine-tuned with this new dataset to become GPT-3.5. (Lynch, 2023)

All GPT models leverage the transformer architecture, which means they have an encoder to process the input sequence and a decoder to generate the output sequence. The model can differentially weight different parts of the sequence to infer meaning and context thanks to the multi-head self-attention mechanism found in both the encoder and the decoder. In addition, the encoder leverages masked-language-modelling to understand the relationship between words and produce more comprehensible responses. (Ruby, 2023)

The model creates a query, key, and value vector for each token in the input sequence. And then, calculates the similarity between the query vector from step one and by taking the dot product of the two vectors, the vector of every other token can be found. This will generate normalized weights by feeding the output of the similarity into a SoftMax function. And generates a final vector, representing the importance of the token within the sequence by multiplying the weights generated by the value vectors of each token.

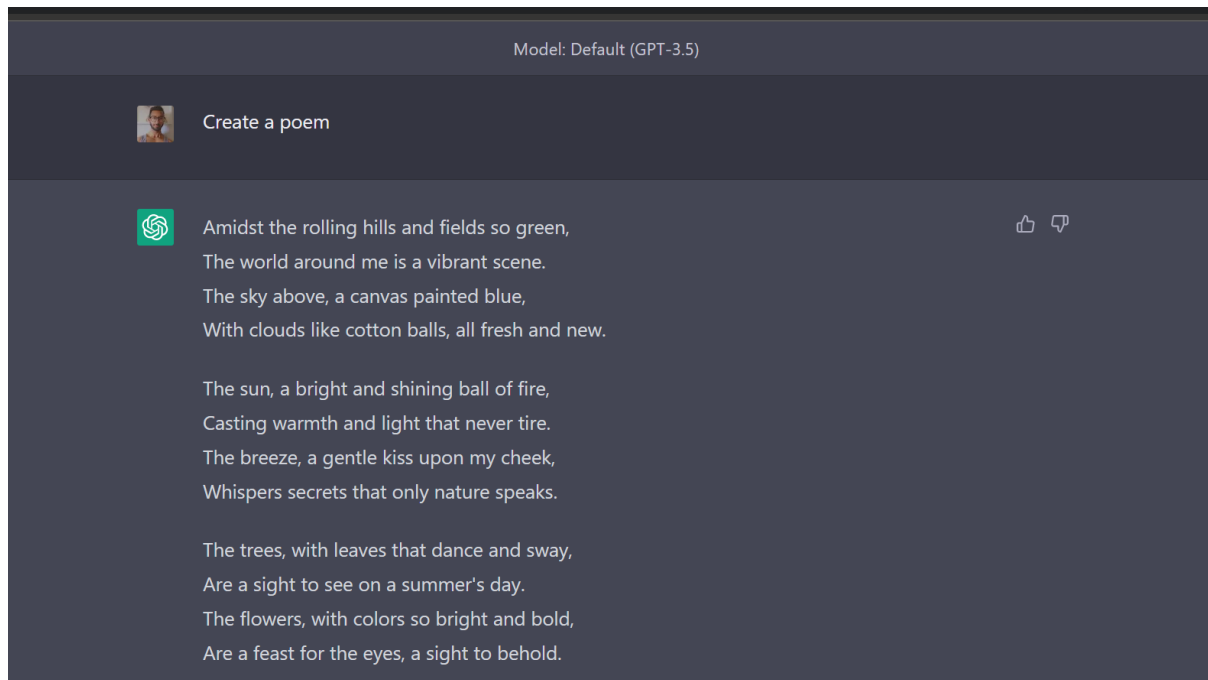


Figure 6: An example of ChatGPT's interface

2.3.4.2 BLOOM

One of the recent achievements in large language models is the open-source model called BLOOM. BLOOM is the first multilingual Model which have been engineer in complete transparency by the largest collaboration of AI researchers. One of BLOOM's functionalities is to generate text in 46 natural languages, 46 dialects and 13 programming languages. It is trained on 1.6TB text data. (KRISHNA, 2022)

BLOOM employs a strategy of predicting the subsequent token in a sentence, based on a set of preceding tokens. This enables BLOOM to connect multiple concepts in a sentence and manage to solve non-trivial problems such as arithmetic, translation, and programming with fair accuracy. BLOOM uses a Transformer architecture composed of an input embeddings layer, 70 Transformer blocks, and an output language-modelling layer. Each Transformer block has a self-attention layer and a multi-layer perceptron layer, with input and post-attention layer norms as demonstrated in the figure 5 below. (Arteaga, 2022)

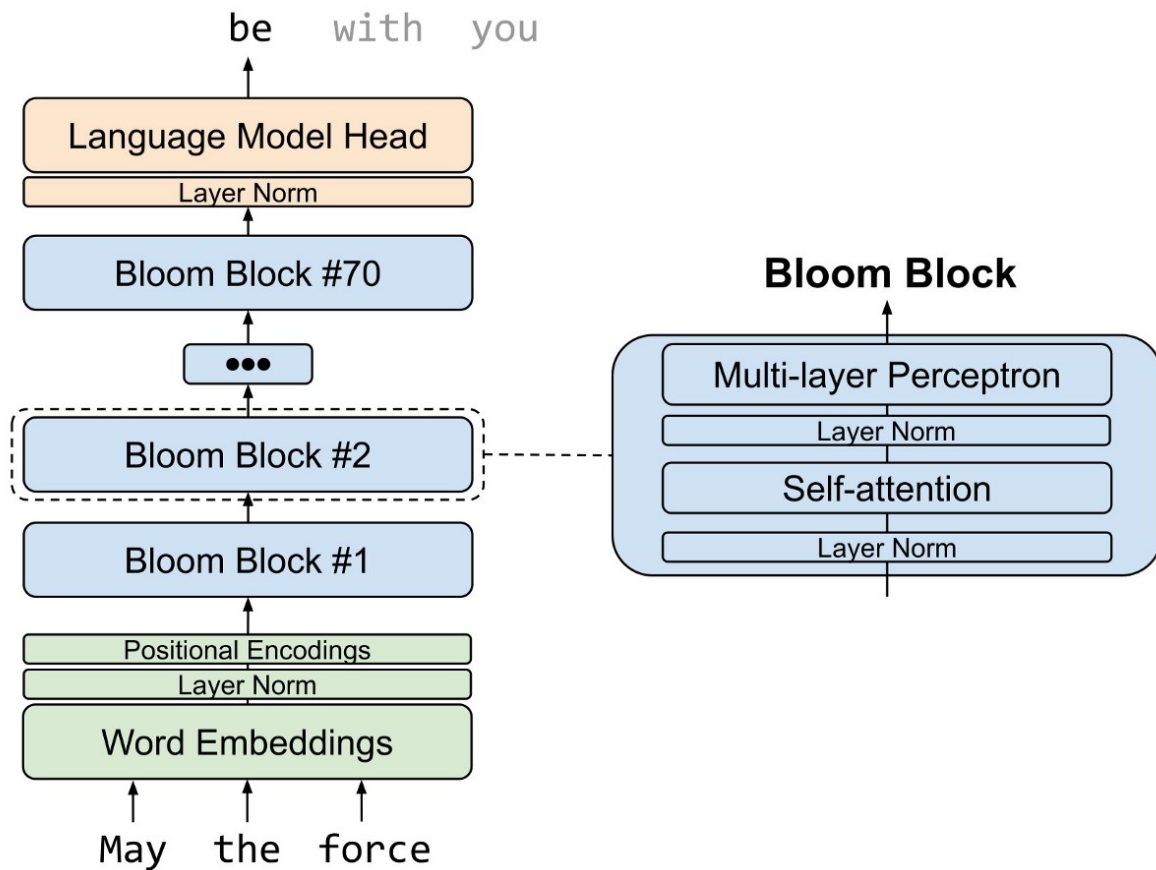


Figure 7: BLOOM Architecture [3]

2.3.4.3 BERT

BERT also known as Bidirectional Encoder Representations from Transformers is a masked language model for natural language processing (NLP) that is designed to help computers understand the meaning of ambiguous language in text from a sentence. As natural language is being processed for computer to understand, most words have the same value thus consider having the same meaning even though two words can be written the same way but have different meaning. For example, bank and riverbank have two different meaning, how the computers will give bank the same value. BERT is using a transformers architecture to overcome this problem and understand the meaning of sentences.

Bert uses a transformers method that consist of word tokenization to covert the sentence into sequence. The word tokenization breaks words like calling into "call" and "##ing". These segments will be feed to find the weight of the sentence through the process of Token Embeddings. (Lutkevich, 2023)

2.4 Neural Networks and Deep Learning

The author will use neural network and deep learning to train a model for the intent classification to help VOYA identifying the command more intelligently. Thus, it is crucial for the author to have a proper understanding of how neural network work at a high level. A neural network is an artificial intelligence technique used to instructs computers to process data in a manner modeled after the human brain. Deep learning is a type of machine learning that employs interconnected neurons or nodes in a layered structure to mimic the human brain. (Zhou, 2022) It develops an adaptive system that computers use to continuously learn from their errors and improve. Artificial neural networks thus make an effort to more accurately solve complex problems, such as summarizing documents or understand the intention of a user's voice command.

A neuron is the basic unit of a neural network that takes some inputs, compute them, and produces one output. For example, the figure 9 (Guarnieri, 2006) below takes 3 inputs to produce one output.

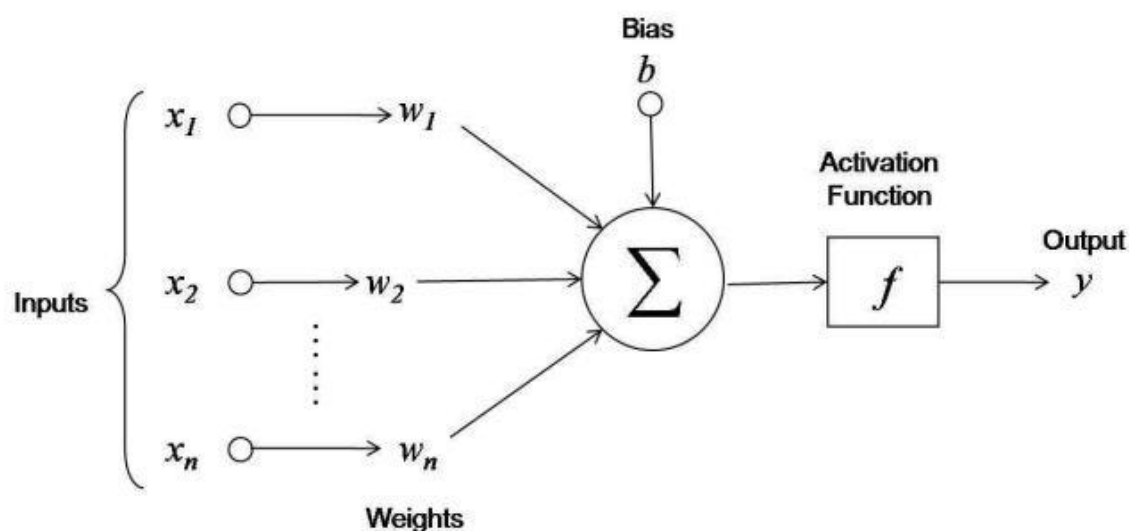


Figure 8: Overview of Neurons in machine learning

The figure above demonstrates how the neurons works. In order to create an output, three processes are in motion. Every input is first multiplied by a weight using the formula below:

$$x_1 \rightarrow x_1 * w_1$$

Figure 9: Formulae to calculate the inputs

The weighted inputs are then added collectively with a bias as shown in the formula below:

$$(x_1 + w_1) + (x_2 + w_2) + (x_3 + w_3) + b$$

Figure 10: Formulae to calculate all the input and bias

Where x are the inputs, w are the weight and b is the bias. Bias refers to error in the machine learning algorithm due to consistently making inaccurate predictions.

The above output would then be passed through an activation function which introduces non-linearity into the network and aids in modeling complex relationships between the inputs and outputs as shown in the formula below:

$$y = f(x_1 + w_1 + x_2 + w_2 + x_3 + w_3 + b)$$

Figure 11: Formulae to calculate the activation function

The output of the layer is created by applying a non-linear transformation to the weighted sum of the inputs and the bias term as inputs. The next layer of the neural network receives this output as input and continues the cycle to train a model, which is known as neural network.

2.4.1 Intent Classification

Intent classification is an essential part and a complex task to achieve in this project. Intent classification is a machine learning task in combination with NLP task which involves determining the underlying intent of a given text or speech input. This task typically involves training a model to find patterns in the input data that are suggestive of intentions. It is typically accomplished using machine learning techniques, such as supervised learning or deep learning. An intent classification will be beneficial to VOYA as it will behave more intelligently as the user give a command.

It can be difficult to implement intent classification, and there are a number of things to take into account to make sure the classification model is precise and accurate. Accurately defining the set of intents is a significant challenge. While avoiding overlap between intents, that is, to gather high quality data, it's critical to identify all potential user intentions. A

poorly defined set of intents may make it difficult for the model to correctly categorize inputs. It is fundamental for the success of this project to research and choose the best machine learning algorithm. There are several components that work together to identify the intent behind an inputted text or speech. These components will be discussed further below. (Anna Esposito, 2021)

2.4.1.1 Tokenization

Tokenization is a crucial pre-processing step in intent classification, which involves separating the input text into discrete tokens, such as words or subwords, which can then be used as features for classification.

The tokenization process typically involves splitting the input text on whitespace and punctuation characters, and then applying additional rules to handle special cases like contractions and hyphenated words. For example, the sentence "Tell me the time" might be tokenized into ["Tell", "me", "the", "time"].

In order to classify content based on intent, these tokens are typically transformed into a numerical representation, such as a term frequency-inverse document frequency (TF-IDF) vector or bag-of-words. The specific machine learning algorithm and architecture being used may have an impact on the representation that is used. (Eckroth, 2018)

It's crucial to remember that tokenization may affect how accurately the intent classification model classifies data. While more complex tokenization techniques may result in longer processing times and greater complexity, inaccurate tokenization can cause incorrect classification. As a result, it's crucial to carefully consider the tokenization strategy employed and how it may affect the precision and effectiveness of the model. (Predicting User's Intent from text using machine learning methods, 2018)

2.4.1.2 Stop-Words

Stop words are frequently used words in a language that, in most cases, are thought to have little to no semantic meaning on their own. The words "the," "a," "of," "and," "or," and "in" are examples of stop words in English.

In order to reduce the dimensionality of the input feature space and increase the effectiveness and accuracy of the model, stop words are frequently eliminated during the tokenization process in intent classification. This is due to the fact that stop words can frequently be found in text but may not offer much information that is helpful for classification.

It's important to remember that, depending on the specific use case and the available data, removing stop words may not always be advantageous or necessary. When stop words are used in conjunction with other words to create phrases that have a particular meaning, they may at times be crucial for accurate classification. The author have therefore decided to not eliminate the stop word as it will not be beneficial to understand the proper intent of a user.

2.4.2 Types of Deep Learning algorithm

It can be difficult to select the best deep learning algorithm for intent classification because it depends on a number of variables, including the type and quality of the input data, the difficulty of the task, and the available computational resources. The type of input data, the quantity and caliber of labeled data available for training, and the level of computational resources needed are some important factors to take into account when choosing an algorithm. Some of the most common algorithm will be cover below with a focus to the implementation of this project.

2.4.2.1 Convolutional Neural Networks

Convolutional neural networks (CNNs) are a subset of the various artificial neural network models that are employed for various applications and data sets. A CNNs is a particular type of network architecture for deep learning algorithms that is used for tasks like image recognition and pixel data processing. Although there are other kinds of neural networks in deep learning, CNNs are the preferred network architecture for identifying and recognizing objects. They are therefore ideally suited for computer vision (CV) tasks and for applications where accurate object recognition is crucial, like facial and self-driving car systems.

The convolutional layer is the fundamental component of a CNN and is made up of a number of filters (also referred to as kernels or weights) that slide over the input data and compute a dot product with a small area of the input data. The output value from each dot product is then added together to create a single output value, which creates a new matrix to represent the convolutional layer's output. The result is a set of feature maps that highlight various facets of the input data. This process is repeated for each location in the input data that the filter can fit. Although the CNNs is an advanced algorithm, it is not suitable for the voice assistant intent classification due to its network architecture for identifying and recognizing objects. (Wu, 2017)

2.4.2.2 Bidirectional Encoder Representations from Transformers (BERT)

Bidirectional Encoder Representations from Transformers (BERT) is a pre-trained deep learning model that can be adjusted for a variety of natural language processing (NLP) tasks, including intent classification. BERT is pre-trained using the tasks of masked language modeling and next sentence prediction on a large corpus of text data. Masked language modeling trains the model to predict the original word from the surrounding context by

randomly masking some words in the input text. The model can learn word context representations through this task. The model is trained to determine whether two sentences are consecutive in the original text or not when it comes to next sentence prediction. This task aids the model in figuring out how sentences are related to one another and can enhance performance on tasks that call for comprehension of sentence-level semantics. (Baranwal, 2020)

BERT can be fine-tuned for particular NLP tasks, like intent classification, after pre-training. The final hidden state of the network is used to represent the input text during fine-tuning after the input text has first been passed through BERT. The final prediction regarding the intent of the input text is then made by passing this representation through a fully connected layer.

While BERT is a highly effective model for intent classification, but it might not be the best option for this project. The author has decided not to use BERT due to the limited computing power allocated to this project as BERT is highly demanding in computing power. BERT has up to 340 million parameters, making it a large and complex model. This implies that it might use a lot of computational power to operate, particularly if you need to adjust the model for a particular task. (Jacob Devlin, 2018)

2.4.2.3 Long Short-Term Memory (LSTM)

Recurrent neural network (RNN) architectures with Long Short-Term Memory (LSTM) are created to solve the vanishing gradient issue that plagues conventional RNNs. Since Hochreiter and Schmidhuber first presented LSTM in 1997, (Hochreiter, 1997) it has gained popularity for a range of natural language processing (NLP) tasks, including intent classification.

The architecture of LSTMs is more intricate than that of conventional RNNs. LSTMs have multiple layers, each with a set of learnable gating mechanisms that regulate the flow of information through the network, as opposed to a single recurrent layer. For modeling long-term dependencies in sequential data, such as text, these gating mechanisms enable LSTMs to selectively remember or forget information from prior time steps. The current word in the input text, along with the output and hidden state from the previous time step, are all inputs that an LSTM uses at each interval. (KIAN LONG TAN, 2022) First, the input is run through a set of trainable weights that determine how much weight to assign to each feature. The previous hidden state is then added to this weighted input, which is then combined with it and sent through a series of gates that regulate how much information is output, forgotten, and remembered.

At every time step, these mechanisms produce a new hidden state and a fresh output that can be used to infer the intended meaning of the input text. The last hidden state can be obtained by passing it through a fully connected layer to get the final output.

2.4.3 Datasets for training model

Data is an essential component of any AI model and the sole reason for the spike in popularity of machine learning today. The availability of data made scalable machine learning algorithms viable as actual products that can bring value to a business instead of being by-products. There are few criteria that the author is taking into consideration when selecting the right dataset for this project. As the intent classification will be in the form of text and not speech, the dataset should be in a textual format. This can include formats such as CSV, JSON, or TXT. The dataset should be in the same language as the target audience for the voice assistant. It is also important to ensure that the dataset covers variations in language such as dialects or regional differences. The dataset should be in the same language as the voice assistant target audience, in this case English as the project is being developed for Ireland's population.

Lastly data quality is important to ensure a high level of output from the training of the model. Data quality will be evaluated by check the dataset for missing values as missing values are prone to inaccuracy ad errors. Alternatively, accuracy will be verify by manually reviewing a sample of the data to ensure that the labels are correct and consistent with the query text. For example, verifying that when the user query "what is the time" or "is it day or night" the intent returns should be "time".

2.4.3.1 Snips dataset

The Snips dataset is a publicly available dataset for natural language understanding (NLU) widely used for building voice assistants and chatbots. The dataset was created by Snips, a company that specializes in developing voice assistants and was acquired by Sonos in 2019.

The Snips dataset consists of queries from a variety of domains, such as music, weather, calendars, and home automation. Each query is labelled with an intent, which represents the user's intention or purpose behind the query. For example, a query such as "What's the weather like today?" would be labelled with the "weather" intent.

This dataset contains 7,248 queries, with an average of 121 queries per intent. Human annotations were performed on the queries gathered from a variety of sources, including forums, customer service logs, and online forums. (Snips Voice Platform: an embedded Spoken Language Understanding system for private-by-design voice interfaces, 2018)

In general, the Snips dataset labels are accurate and consistent, which is important for training a high-quality intent classification model. In order to assign the labels to the data, human annotators were trained to make sure that the labels were accurate. (snips_built_in_intents, 2023)

2.4.3.2 CLINIC OOS-EVAL dataset

"oos-eval" is a publicly available dataset for evaluating out-of-scope (OOS) intent detection models. CLINC, a research group at the University of Illinois at Urbana-Champaign that develops conversational AI systems, created the dataset. A collection of queries from various domains, including both in-scope (IS) and out-of-scope (OOS) queries, make up the "oos-eval" dataset. The IS queries are queries that fall within the intended domain or scope of the voice assistant, while the OOS queries are queries that fall outside of the intended domain or scope. The goal is to evaluate the ability of the intent detection model to correctly identify OOS queries and handle them appropriately. For example, if there is an out-of-scope query, the assistant should direct that queries to the chatGPT API as the function to match this query is not yet created.

The dataset is available in JSON format and contains a total of 13,083 queries, with 11,888 IS queries and 1,195 OOS queries. The queries were collected from multiple sources, including online forums and customer service logs, and were labelled by human annotators, therefore less room to missing values or data inaccuracy. The "oos-eval" dataset is constructed to be challenging, with a high degree of flexibility and complexity in the queries. This diversity contributes to the robustness of the intent detection model and its ability to respond to a variety of user queries. (Stefan Larson, 2019)

2.4.4 Chosen Deep Learning algorithm and dataset

While CNN and BERT are more frequently employed for image and general NLP tasks, LSTM is specifically designed to handle sequential data, such as text. As command from voice input frequently involves long-term dependencies and variable-length input sequences, LSTM is a natural choice for intent classification due to these characteristics.

Additionally, BERT is a highly effective model for intent classification, but to optimize its pre-trained weights, it needs a lot of labeled data. In contrast, LSTM is a more effective option for small and medium-sized projects because it can perform well with smaller labeled datasets. Therefore, the author opted or the LSTM algorithm for training the model.

As both datasets are highly effective in training and intent classification model with high data quality and accuracy, the author opted to use the CINIC oos-eval dataset as the dataset can handle out of scope queries as well as in scope queries, which will be beneficial for the future phase development of the project.

2.5. Programming Languages researched

There are many programming languages that could be suitable for the building of a voice assistant. This section will be discussed between the three most relevant programming language to build the voice assistant.

2.5.1 JAVA

Java is a highly popular tool for creating web development and desktop projects. JAVA can be run smoothly on a variety of platforms without recompiling virtual machine technology. This programming language has all the essential features for an improved interface, allowing it to perform powerful voice assistant building in Java. The standard widget and language visualization function can additionally help a programmer to manage the work regularly. Furthermore, it is also very practical and easy to code due to its entire and voluminous libraries.

However, Java has some shortcomings. They are based on the fact that the programming language is slower, which means more response time is required. As far as AI is concerned, this is still an imperfect tool, so third-party support may be necessary to make a voice assistant in Java. Nonetheless, it is worth noticing that it is a productive and reliable development tool. (Flannagan, 2005)

2.5.2 Lisp

Lisp was purposefully created as a programming language for projects with artificial intelligence. That is why it is excellent for creating AI projects. Lisp came out in 1958 by John McCarthy and is still a robust tool. It is a powerful programming language as it went through fundamental standardization for military and industrial use. (Thomson, 2011)

Building a voice assistant with Lisp will allow to solve certain problems quickly owing to the language's adaptability. Teaching, designing, and developing the multilingual capabilities of AI can be done with One-off modelling and prototyping. Lisp has a rich library which process emblematic information quickly enough in providing a faster response. Lisp is widely used for inductive logic and machine learning challenges. (Norvig, 1992)

However, Lisp is no longer the primary tool for building AI anymore as its syntax is quite complex. And this is a big drawback in comparison, with Python. Nevertheless, it can be used as an additional tool to perform prototyping of project.

2.5.3 Python

Python is a robust programming language centered on enhancing developer's productivity and code readability. It is a general-purpose language, which means that it can be used in various programs as well as solving many problems. Python has AIML, an artificial intelligence markup language which enable an easier way to create product syntax. It also has PyNLPI libraries among a huge library for building language models. Natural Language

Toolkit, which was originally written in Python and is the prototype of modern NLP. Python has an active community based around its different libraries. Therefore, help is available when running into issues. (Eckroth, 2018)

2.5.4 Chosen Programming Language

Python is ideal because it has an extended library to support the development, has simplified integration with other parts of the project that use machine learning, support Dynamic and scalability of code.

Thus, it turns out that the voice assistant building in Python is a profitable and affordable way to help achieve project goals. This programming language has an ever-growing community, which lets developers rapidly receive support and advice regarding the usage of its libraries and Python itself.

2.6. Python libraries Researched

Python was chosen as the programming language to be used among the others mentioned. The library in python is vast and provides different packages. And all the libraries will essentially provide the desired output when implemented. However, which library to use and why. This section will discuss on the different libraries available to deliver the end product and the one that is more suitable.

There are few main areas to focus on when building the voice assistant as follows:

1. Speech Detection,
2. Convert text to speech,
3. Speech to text,
4. A process to allow the leverage of external application,
5. The graphical user interface.

2.6.1 Speech Detection

This section will cover all the libraries that are needed to successfully build VOYA and its components.

The first step to building VOYA is to have a speech recognition library to detect speech input from user. Speech should be translated from physical sound to an electrical signal with a microphone, and then to digital data with an analog-to-digital converter. Once digitized, various models can be used to transcribe the audio to text.

There are multiple packages for speech recognition exist on PyPI which include:

- assemblyai
- SpeechRecognition

2.6.1.1 AssemblyAI

AssemblyAI's API can be used to transcribe and understand audio/video files with pre-build AI models. The AssemblyAI CLI is the easiest way to test API. It is simple to install on any operating system and works on virtually any audio or video file. AssemblyAI implement different language support recognition and also include profanity filtering. AssemblyAI would be an ideal Api to use for VOYA voice detection. However, it is a paid API and will process on the number of voice input to be charge at. Therefore, further research has been dropped for AssemblyAI.

2.6.1.2 Speech recognition

The SpeechRecognition library acts as a binder for numerous well-known speech APIs and therefore is extremely flexible. One of these include the Google Web Speech API which supports a default API key that is hard coded into the SpeechRecognition library. That means it will allow the development of the project without having to sign up for a service. The flexibility and ease-of-use of the SpeechRecognition package make it an excellent choice for VOYA project. (Ravikiran, 2023)

2.6.2 Convert text to speech

The second step to building VOYA is have a *Convert text to speech* library to convert the detected speech input from user. The Speech to Text transcribes audio to text to enable speech transcription capabilities for applications.

There are multiple packages for speech recognition exist on PyPI which include:

- IBM Watson Studio
- C

2.6.2.1 IBM Watson Studio

IBM Watson Studio is an integrated environment intended to develop, manage models, train, and deploy AI-powered applications. It is a Software as a Service (SaaS) solution delivered on the IBM Cloud. The IBM Cloud offers an abundant of services like Speech-to-Text, Text-to-Speech, Visual Recognition, Natural Language Classifier, Language Translator. However, it is a paid API and will process on 500 minutes use per month. Therefore, further researched has been drop for IBM Watson Studio.

2.6.2.2 pyttsx3

pyttsx3 is a speech to text and text-to-speech conversion library in Python. It operates offline and is compatible with both Python 2 and 3. The pyttsx3 module supports two voices type which is female and male. It supports three TTS engines: (Sidiq, 2023)

- sapi5 – SAPI5 on Windows
- nsss – NSSpeechSynthesizer on Mac OS X

- espeak – eSpeak on every other platform

2.6.3 Graphical User Interface

This project will involve a graphical user interface (GUI) to provide visibility to a dialog between the user and the voice assistant. The GUI will be designed as a minimalistic small window located at the bottom-right of the screen above the taskbar. The GUI will entail a simple dialog box with the assistant logo and controls to close or minimise.

There are several options on creating the GUI through software as a service application or UI libraries tool. This section will cover on the top three beneficial option.

2.6.3.1 Windows Presentation Foundation

Windows Presentation Foundation (WPF) is a Microsoft .NET GUI tool which is the most popular and modern choice for developing Windows OS Desktop applications.

WPF has incredible templating, styling, and binding features. In addition, there are lots of external tools/libraries such as Model-View-View Model Light, Prism, Caliburn. Micro and control suites from companies like Telerik, which make the development process far more manageable. (Wangari, 2023)

However, it has a rather sheer learning curve which entails learning a syntax called Extensible Application Markup Language (XAML) and also, a design pattern called Model-View-View Model (MVVM).

2.6.3.2 Tkinter

Tkinter is an open-source Python Graphic User Interface (GUI) library which provide simplicity of use. As the author does not have a lot of experience with python GUI, Tkinter provide a strong position for beginners and intermediates to begin with. Tkinter is pre-installed with all new version of python. Tkinter provides a vast collection of well-known widgets, such as in-box option, buttons, labels, checkboxes. (Amos, 2022)

The Tkinter widgets are good, however, it does not have a native look and feel. Tkinter are limited on the widgets and does not provide advanced widgets.

2.6.3.3 PyQt

PyQt is an open software for graphical user interface (GUI) used as python module. It is extracted from the library of Qt. PyQt is the result of the combination of the Qt library as well as the Python language. PyQt is coming with the Qt Builder. PyQt will be used to get the python code from the Qt Creator through the support of the Qt designer. (Ramos, 2022)

PyQt supports numerous UI components for instance Qt provides multiple widgets, such as buttons or menus, all designed with a basic interface for all compatible platforms.

2.6.4 Deep Learning

When building and training LSTM models for intent classification, typically a combination of multiple Python libraries are used. There is not one single python library that can achieve all the component in training the model. The various capabilities and functionalities offered by these libraries can be combined to produce a thorough workflow for developing and refining the LSTM model.

2.6.4.1 Scikit-learn

Scikit-learn (or sklearn) offers a variety of algorithms and tools for classification, regression, and clustering. Scikit-learn offers support for creating and training straightforward LSTM models for intent classification, despite being primarily a library for conventional machine learning tasks.

The dataset will be divided into training and testing sets using scikit-learn. The split function will divide the input data and labels into training and testing sets randomly and according to a predetermined proportion. The categorical labels (intent names) will be transformed using scikit-learn into numerical values that the neural network can use as targets. String labels are converted into integer labels using the LabelEncoder class, and integer labels are transformed into one-hot encoded vectors using the OneHotEncoder class. The neural network will be trained using the one-hot encoded vectors as its targets. (Gupta, 2023)

2.6.4.2 Keras

Keras is a high-level deep learning library which offers a simple user interface. It was created with the intention of facilitating deep learning for researchers and practitioners, especially those who are not machine learning experts. With only a small amount of coding needed, Keras enables users to define and train neural network models. In addition to feedforward networks, convolutional neural networks (CNNs), recurrent neural networks (RNNs), and combinations of these, it supports a wide variety of neural network architectures.

Keras will be used to tokenize text data means dividing it up into individual words or tokens. It will also be used to pad token sequences to a set length, ensuring that each input sequence is the same length. It will also be used to specify the model's LSTM layer. This layer is used to model the word order in the input text and to record word relationships. It will also be used to specify the model's sequential architecture, or the order in which the layers are stacked.

2.6.4.3 Tensorflow

Tensorflow is an open-source platform created by Google and is now widely used for a range of deep learning and machine learning tasks by researchers and developers all over the world. TensorFlow's capacity to efficiently carry out numerical operations using specialized data structures known as tensors is one of its main advantages. Images, audio signals, text, and other types of data can all be represented using tensors, which are multidimensional arrays.

2.6.5 Application Programming Interfaces - APIs

Application Programming Interfaces, known as APIs, are a set of protocols, routines, and tools for building software applications. An API specifies how software components should interact with each other and provides a standardized way for applications to communicate and exchange data. In the scope of this project, APIs will act as the backend of some user's queries to communicated back to the assistant. This will enable the author to integrate the different functionalities to create more complex and interconnected applications. The functionalities that requesting API are one of a kind and therefore only one platform available to request it from. VOYA will leverage the use of three APIs respectively for ChatGPT, generating an image, and getting weather data. As discussed in the above in the large language model section, ChatGPT fall in the scope of this project and therefore an API is needed to avail of this service. Additionally, an API will be needed for getting up to date weather data from visual crossing platform providing a free service compared to another platform. Lastly an image generation API is required from dreamwork platform to allow VOYA to generate images on the go as user requested.

2.6.6 Chosen libraries

As discussed above for the different libraries available for the development of Voya. The author believes using speech recognition library is best suited for recognising voice input as it has a huge community in python and provide full access to functionalities for free.

The pptsx3 library is chosen for the text to speech and speech to text conversion as it provides an easy implementation with python on the level of transcribing voice input as opposed to the other which only provide minimal functionalities as they are paid software as a service.

PyQt is chosen as the graphical user interface toolkit compared to Tkinter and WPF as PyQt have a more modern approach. And also, offer an intensive library of widget which provide the necessity to test for accessibility as it is being designed.

2.7. Hardware Requirements

As this voice assistant is for the purpose of facilitating task within a user's own desktop machine. The first hardware requirement would be a desktop or laptop that leverage windows operating system. Secondly, that machine needs to be equipped with a microphone to receive user input through voice command and a speaker to output the results of a commands. Alternatively, a user can use a headset as most headset nowadays have built-in microphone. At a later stage of the future version 2 of VOYA, the user may require a keyboard to facilitate typing as an input for a command.

2.8. Existing Final Year Projects

2.8.1 Title: NLPurchase – eCommerce Chatbot

Student: Stephanie Finn

Description (brief): This project implements chatbots in commerce in order to improve benefit in commerce, and what are the current limitations of chatbots to be used as a natural language interfaces.

What is complex in this project: Implementing a chatbot with Natural language processing tends to be hard as to provide that human side to the chatbot. This technology is well implemented in private companies, but the implementation guide is not made available to the public.

What technical architecture was used: JAVA, natural language processing APIs, mangoDB

Explain key strengths and weaknesses of this project.

Weakness: It is carried in the waterfall framework and therefore does not accommodate for changes.

Strength: It will provide a market leading solution to commerce to increase benefit as chatbot is still new in the market.

2.8.2 Title: AAC Using Natural Language Processing

Student: Mark Naylor

Description (brief): AAC provides people with little or no speech a means to communicate and function in their daily lives. It will present various boards of symbols to the user that can be tapped to form sentences and it will speak the words and phrases that the symbols represent on clicking them, using text-to-speech.

What is complex in this project: Implementing a model view controller can help with security within code but can be hard to implement.

What technical architecture was used: XCode, Swift, natural language processing, Model view controller, PHP MySQL.

Explain key strengths and weaknesses of this project.

Strength: Prediction that is based on a user's comprehension level is a great way to offer the user with quick selection of the next words.

Weakness: To implement customisable communications board with symbols can prove to be hard as it needs to have a dataset with some learning models before the correct output is shown.

2.9. Conclusion

For this conclusion, as there are many technologies out there to implement a voice assistant, conducting intensive research on the technologies have provide the author with certainty on the best approach to choose. From the research on technologies, the author has decided to design a minimalistic GUI similar to Apple's SIRI to limit complicated interface. A vast option of programming language is available such as lisp or java. However, the best option is python as it has a substantial library that can be implement in this project. The challenge to overcome the gap between human and computer has been identify.

3. Software Design

3.1. Introduction

This section will describe the software design guidelines and best practices used in the development of VOYA, as well as the various functionalities and approaches utilized, such as task-oriented and knowledge-oriented approaches. Additionally, this segment will offer a comprehensive testing plan for VOYA, including continuous monitoring, manual-based testing, usability performance testing, and unit testing, along with specific test cases and evaluation methods. Finally, the section will provide an analysis of the VOYA command structure, including the wake word, request, and expression, which is critical to ensuring accurate voice recognition and effective system response. The results are expected to contribute to the ongoing development of VOYA and provide insights into the design and architecture of the system for future research and development.

3.2. Software Design

The software design of VOYA incorporates several technologies to provide an effective voice assistant. As voice assistant are relatively new to the market and is still in research phase, there have not been much software design guidelines to follow as a standard. Microsoft developers have provided a list of best practices for designing voice assistant scope for

Cortana. For the purposes of the design concept, the author will follow the main guidelines lay out by Microsoft: (Design assistant experiences for Windows 10, 2022)

- i. Provides UI indicators for listening states.
- ii. UI adapts as window sizes change.
- iii. Detect lock state and request activation.
- iv. Do not provide always persistent UX that would block access to the Windows lock screen.
- v. Voice activation preview
- vi. Detect unlock state and request background activation.
- vii. Draw minimal listening UX in the preview pane.
- viii. Draw a close X in the top-right and self-dismiss and stop streaming audio when pressed.
- ix. Resize or hand-off to the main assistant app view as needed to provide answers.

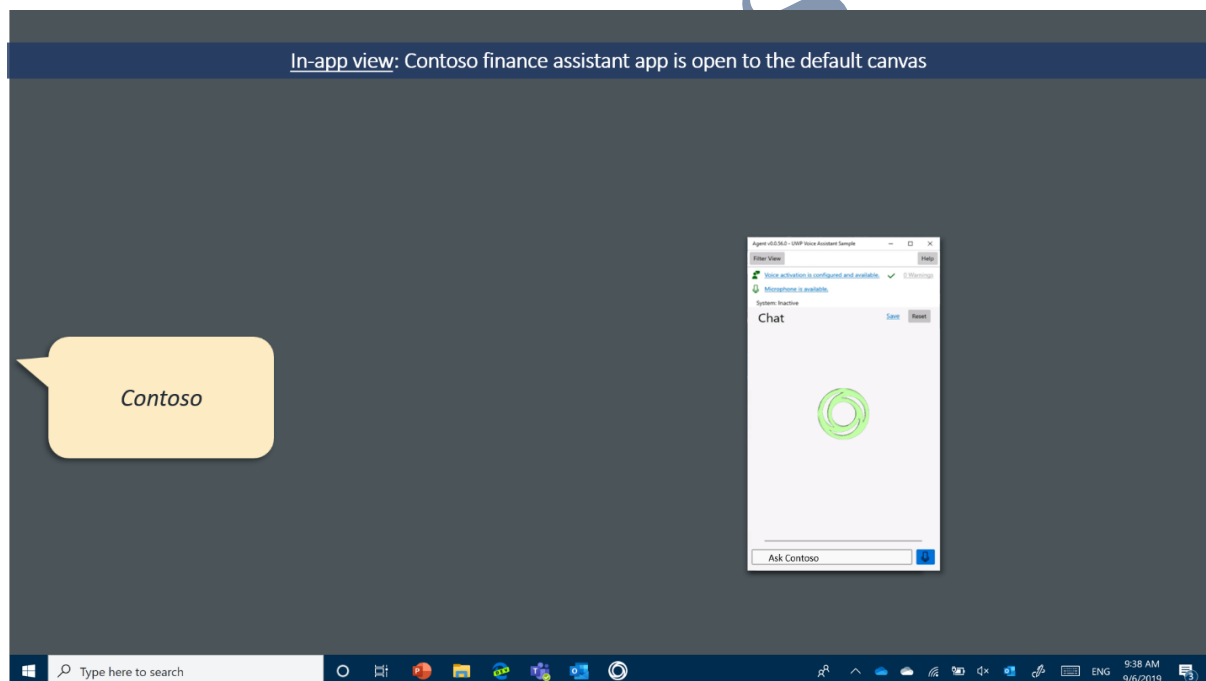


Figure 12: Microsoft In-app view. Initial state when voice activation begins [10]

The prototype development of VOYA will be discussed on the implementation of the task-oriented approach features, for example, opening an application and getting date and time. The development of the prototype is carried out on Figma tool. Figma is a cloud-based prototyping tool to create high level fidelity prototype with an easy integration to target each component individually.

3.2.1. High Level Fidelity prototype

A high-level fidelity prototype was designed in order to pinpoint the different component required and provide a visual aid to the implementation of VOYA and its expecting outcome.

This has allowed to get detailed feedback on certain elements of the design that would not be possible with low level fidelity.

An overview of a high-level fidelity of the end product implementation. The figures below demonstrate the simple and easy interface while it is ask to open Microsoft work with voice command.

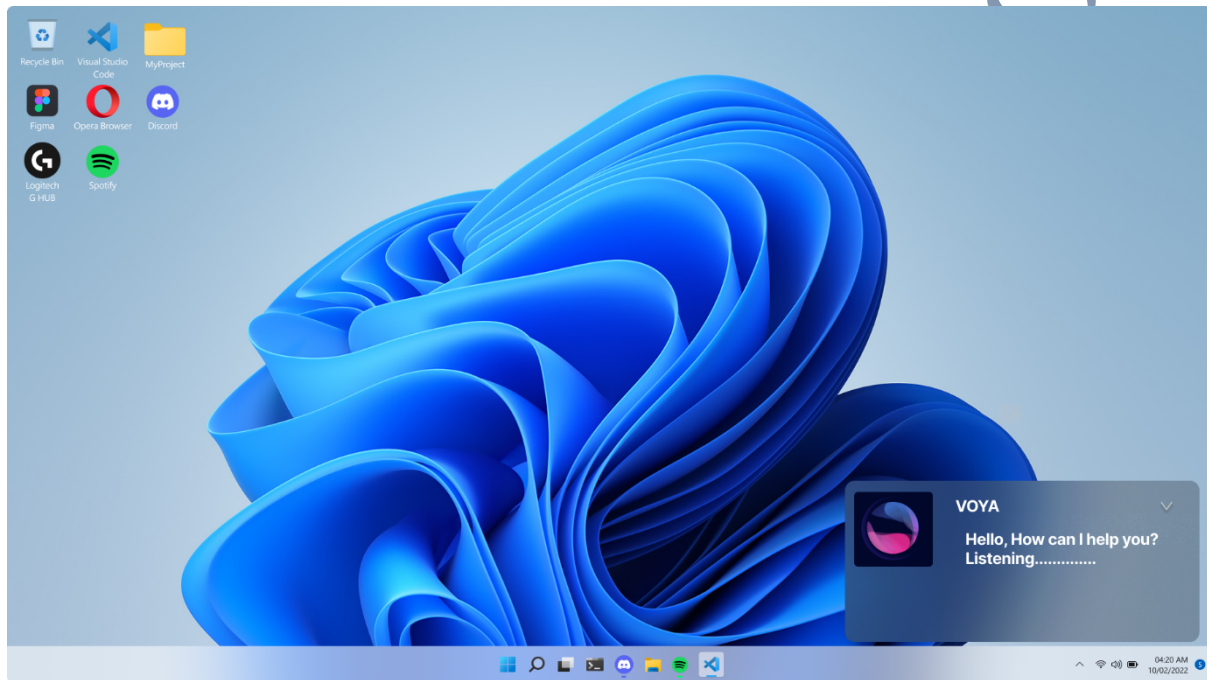


Figure 13: High-level fidelity prototype of VOYA

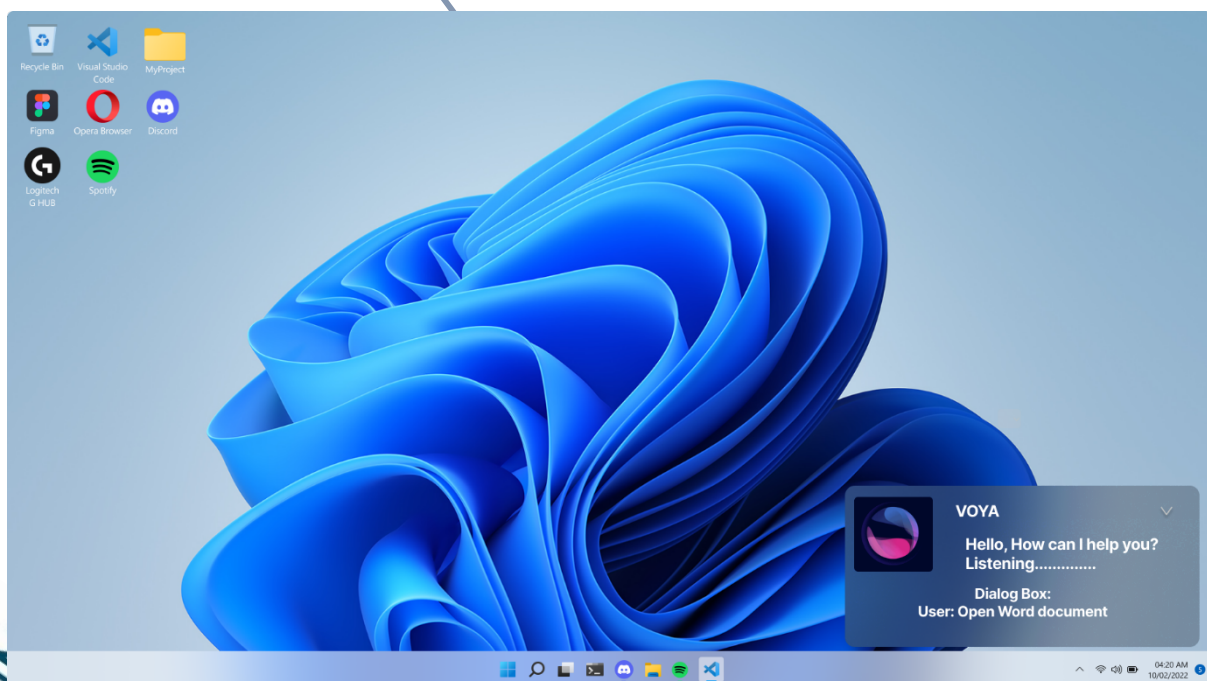


Figure 14: High-level fidelity prototype of VOYA

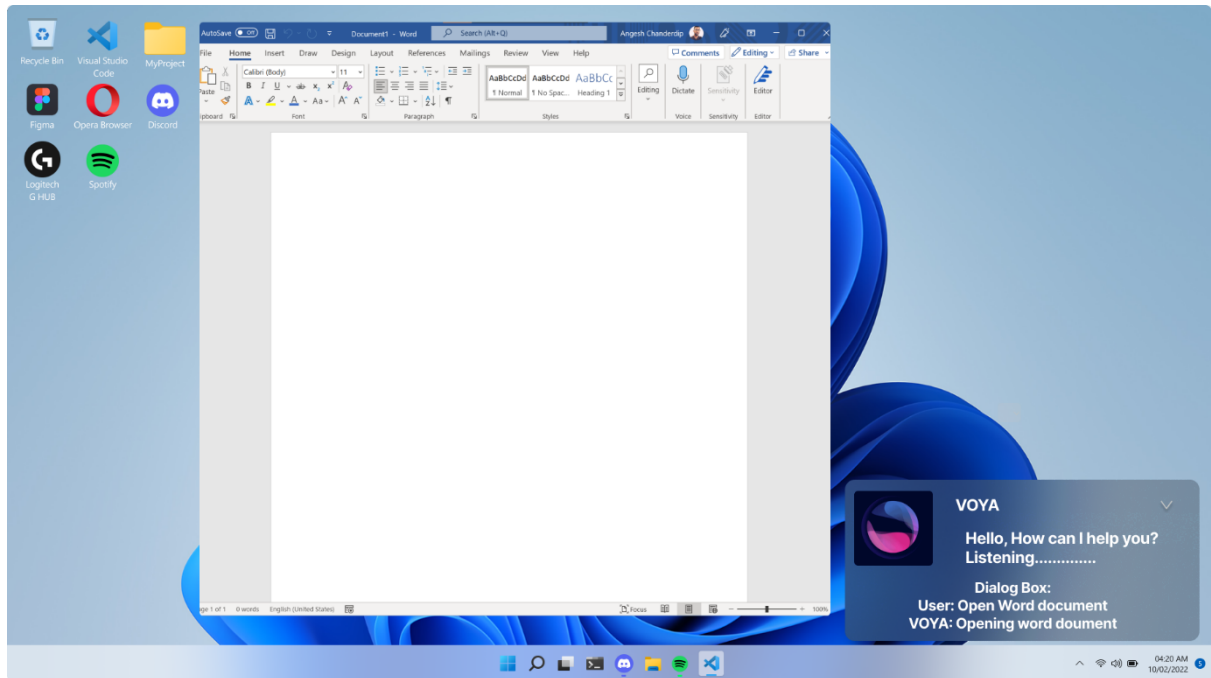


Figure 15: High-level fidelity prototype of VOYA

3.2.2 Use case diagram

Open and close an application.

VOYA will listen for the wake-up word “Hey Voya” to actively listen to user command for example, “Open word document”. VOYA will process the command for the word clarity using the PYTTS libraries and compare the command to the highest appropriate output.

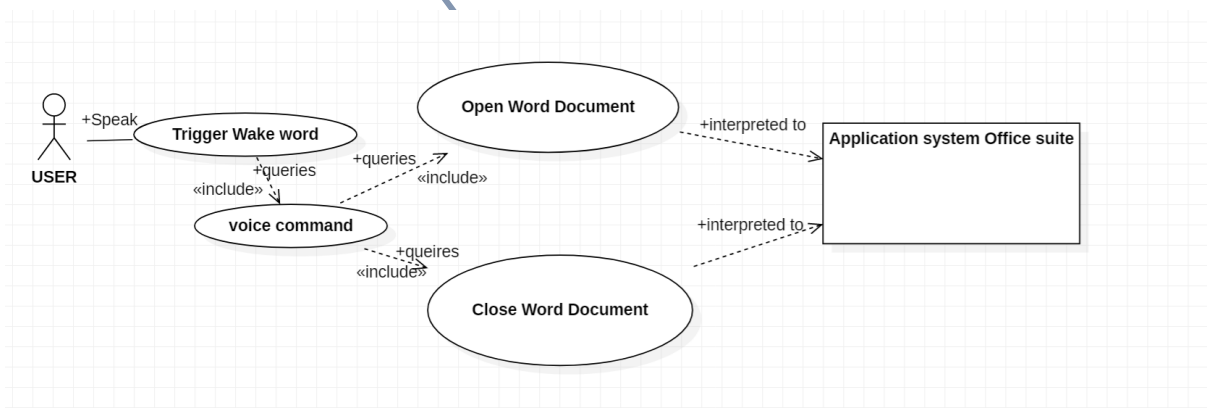


Figure 16: Use case diagram of Opening Word document

Getting date and time

VOYA will listen for the wake-up word “Hey Voya” to actively listen to user command for example, “What date is it. Or what time is it?”. VOYA will process the command to check for word clarity by means of the PYTTS libraries with comparison to the created library

date_time.py and acquire the current date and time as a voice output as well as a visual text on the interface.

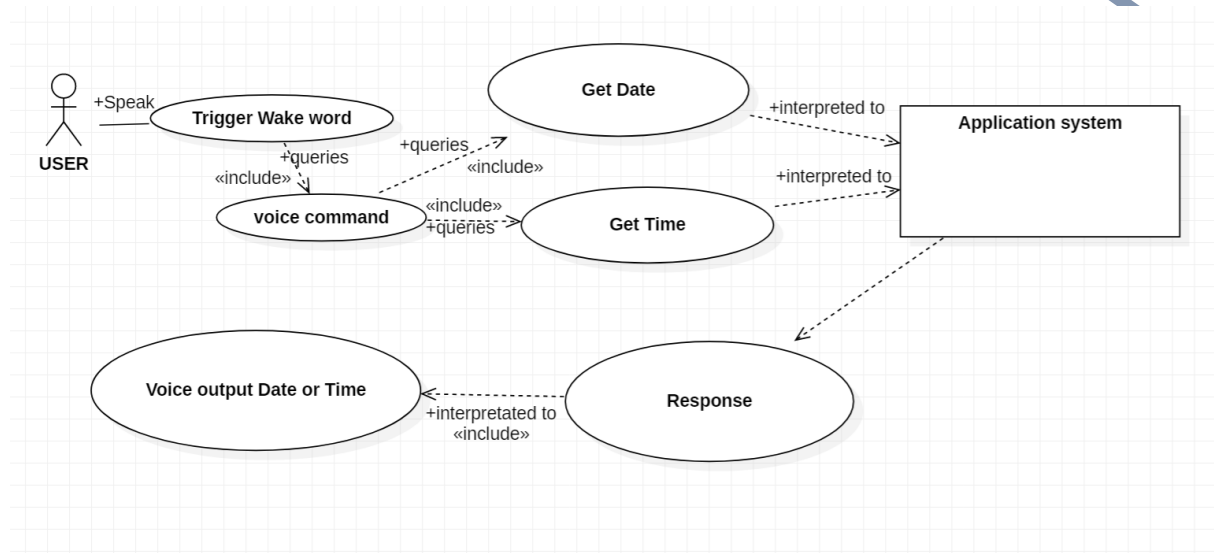


Figure 17: Use case diagram of Getting date and time

3.2.3 Class diagram

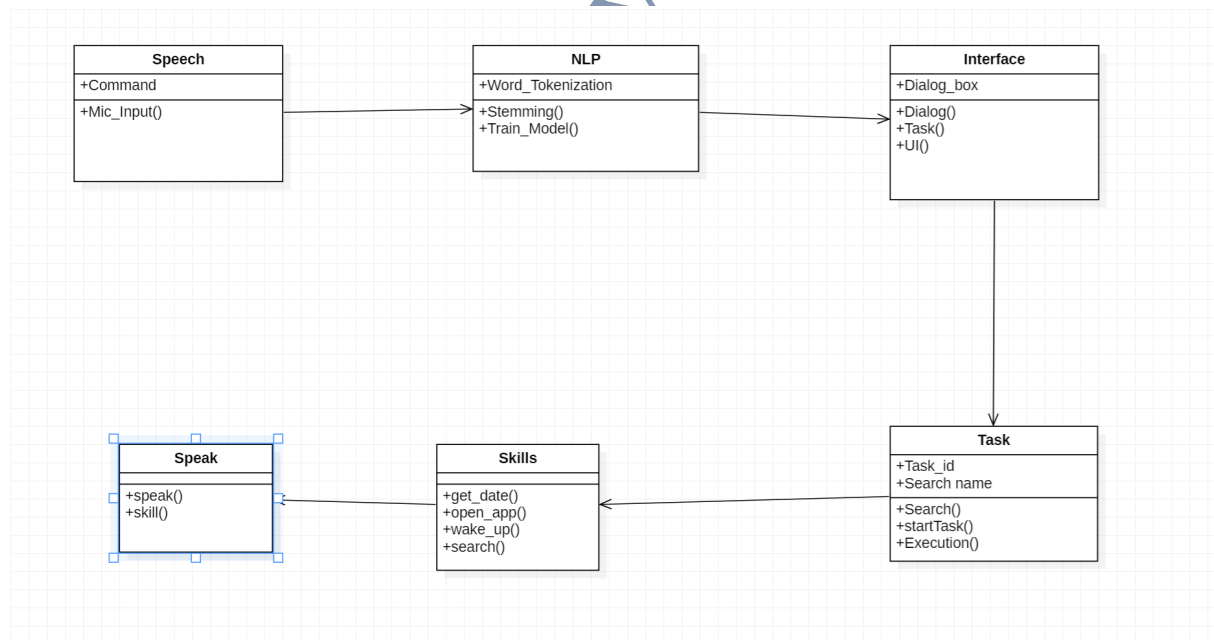


Figure 18: Class diagram of VOYA

3.2.4 Sequence diagram

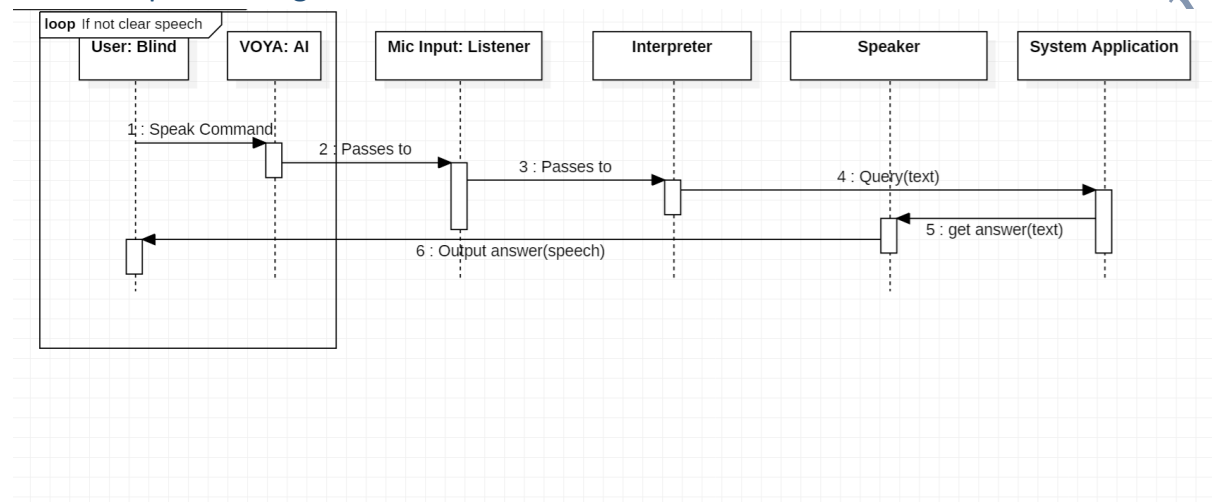


Figure 19: Sequence diagram for Task Oriented Approach

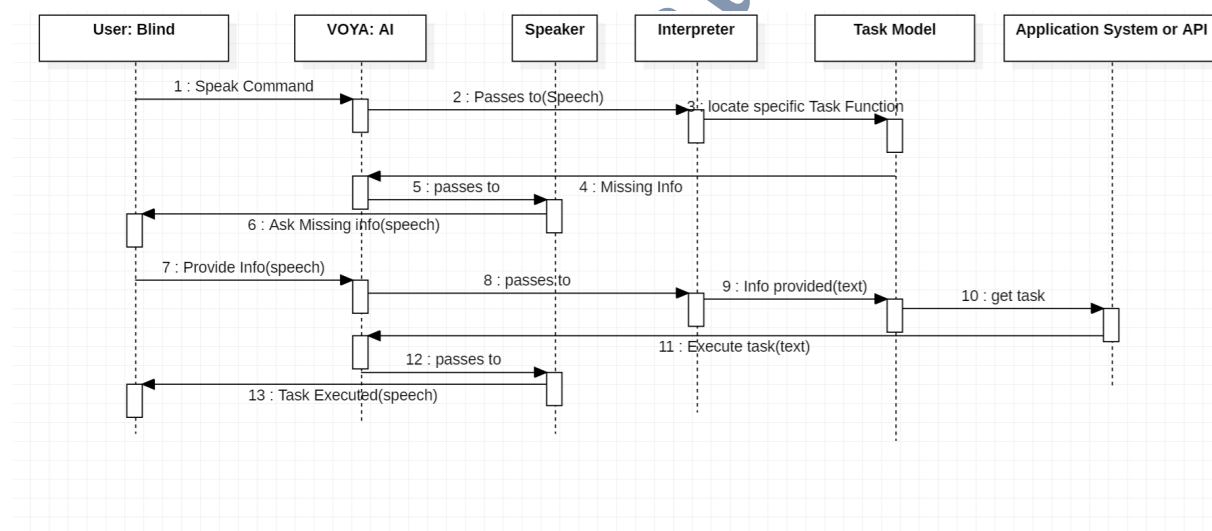


Figure 20: Sequence diagram for Task Oriented Approach

3.2.5 Activity diagram

An activity diagram is an extremely useful tool in UML as it helps modelling workflow by using activities. It details the flow of one activity to another within VOYA. Activity diagram also help with high level understanding of the system's functionalities and investigate business requirements at a later stage. The activity diagram outlined below illustrates the flow of events on how VOYA execute a task.

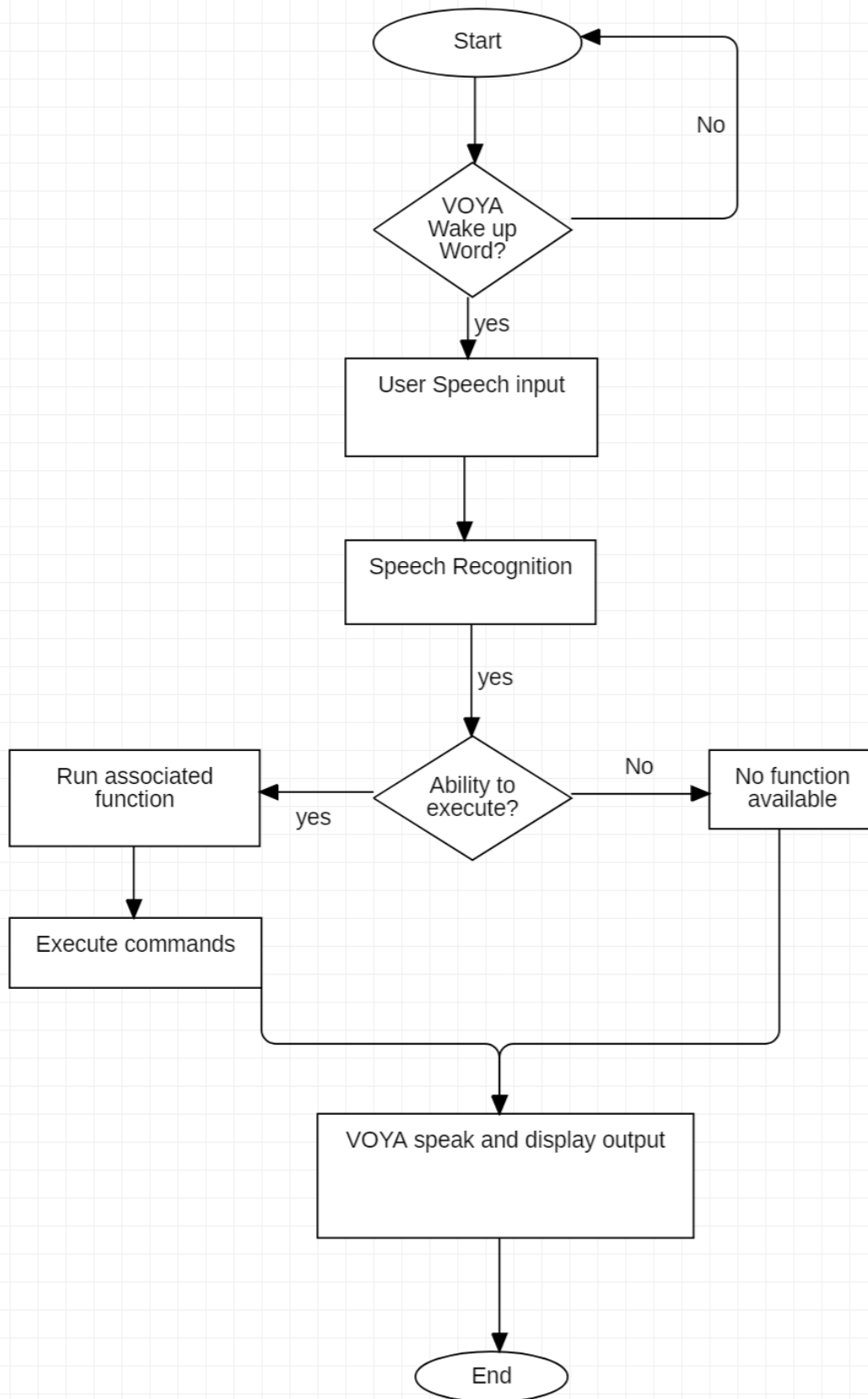


Figure 21: Activity diagram of VOYA executing a command

3.2.6 Software Methodology

“Project management is the practice of applying knowledge, skills, tools, and techniques to complete a project according to specific requirements”. (PMI, 2015) Project management is a crucial part in achieving goals and following through with plans and expectations. However, getting your team organized and deliver the project in time, budget and quality is often challenging. Following a project management methodology can help a project manager organize the project into an organised, streamlined process. A team collaboration become more efficient, and projects become better organized with a PM methodology. (PMI, 2015)

There are many different types of Project Management Methodologies. The following table will present the top 3 project management methodologies consist of the traditional sequential methodologies and the agile family methodologies that might be used to successfully deliver the end-product.

3.2.6.1 P.M.B.O.K methodology

Methodology 1	
Name:	P.M.B.O.K methodology
Definition:	PMBOK stands for Project Management Body of Knowledge, and it is a document containing processes, best practices, terminologies, and guidelines that are accepted as standards within the project management industry. (PMI, 2015)
Description:	According to project management institute, PMBOK acknowledge five basic process groups and ten knowledge areas typical of almost all projects. The concepts are applicable to projects, programmes, and operations. The five processes are: 1. Initiating 2. Planning 3. Executing 4. Monitoring and Controlling 5. Closing
Pros:	1. Provides documentation PMBOK involves processes and methods that are documented each step of the way which helps knowing what processes are effective in getting better results in

	terms of project management. 2. A guide for Project Manager PMBOK is a guide to knowledge and good practices created by project management professionals who update it regularly which is available to project manager for project's success. 3. Standardized PMBOK will allow to standardize practices across departments. This means that the people in development manage projects in the same method as those in distribution. (Bowen, 2020)
Cons:	1. Only project managers are allowed for decision making in PMBOK. 2. It is difficult for handing over aspects of the management to other parties and senior managers. 3. It has too many processes if it applied in small projects.

3.2.6.2 Kanban methodology

Methodology 2	
Name:	Kanban methodology
Definition:	Kanban is a visual system for managing, design and improve work as it moves through a process. Kanban provide a visual on both the process and the actual work passing through that process. The objective of Kanban is to recognise possible blockages in your process and fix them so work can flow through it cost-effectively at an optimum speed or quantity.
Description:	In a Kanban system, Kanban cards play an important part. These cards are the key tool that designate the development of work for a company, movement of

	inventory for a manufacturing line and personals assigned to a task or service. It maintains the tasks which form part of an explicit operation of the corporate. All the related resources and employees are continuously informed about the progress that can trigger or delay the activities based upon the levels of completion. A Kanban card consists of following heading: • Backlog • To do task • Work in progress • Completed
Pros:	1. Kanban system benefit to the direct cost reduction and wastage reduction of the company. Kanban system improves the flow and management of inventory by directly supporting the corporate to pursue the company's existing systems that reduces carrying or holding costs and makes sure smoother running of inventories. 2. Kanban not only consists of manual guidelines or cards but also visualizations of the process outputs which makes the review of work easier. This might also underline other possible problematic areas where additional attention is needed. 3. Kanban system is a very responsive system and does not promote any lags or delays. As the tasks are continuously shifted between the columns of the Kanban cards which automatically highlights the areas where attention is needed.
Cons:	1. Kanban is not usable as an independent tool. It is not a methodology that could be applied solely rather it can be merged with other processes. 2. As the tasks are continuously shifted between the columns of Kanban board, the prediction of specific timelines for completion of tasks or activities becomes difficult.

	3. Kanban will become very difficult to apply if too many activities or tasks are interrelated in a system. This is because such systems enhance the possibility of transfers of goods and expertise amongst different tasks too often and increases difficulty to keep the pace of all these activities.
--	---

3.2.6.3 Scrum Methodology

Methodology 3	
Name:	Scrum methodology
Definition:	Scrum is an agile project management methodology or framework used principally for software development projects with the goal of delivering new software capability every 2-4 weeks. It follows the values and principles of agile, but includes further definitions and specifications, especially regarding certain software development practices to develop higher quality faster. (What Is Scrum Methodology?, 2020)
Description:	Scrum relies on a self-organizing, cross-functional team. The scrum team is self-organizing, there is no overall team leader who takes decision. Scrum teams are sustained by two specific functions. Firstly, a Scrum Master, who can be assumed of as a coach for the team, helping team members use the Scrum process to perform at the optimum level. Secondly, the product owner (PO) which represent for the company, customers or users.
Pros:	1. Scrum sprints: The Scrum method is profoundly focused on sprints where the project team breaks down a Wishlist of end-goals into small pieces, then works on them in 30-day sessions with daily stand-up meetings. Therefore, makes it easy to operate large and complicated projects. 2. Fast paced: The sprint approach with its 30-day limit and daily stand-up meetings promotes rapid iteration and development. 3. Team-focused: Since the project team is expected to manage itself, Scrum teams have clear visibility into the

	project. That imply project leaders can place their own priorities as per their own knowledge of their capabilities. (Cohen, 2019)
Cons:	1. Scope creep: As there is no fixed timeline, nor a project manager for scheduling and budgeting, Scrum can easily lead away from the scope. 2. Higher risk: Since the project team is self-managing, there is a higher risk of failure unless the team is highly disciplined and motivated. Scrum has a very high chance of failure, if the team doesn't have enough experience. 3. Lack of flexibility: The project-team focus means that any resource leaving the team in-between will hugely impact the net results. This approach is only flexible for a small team and not for large teams. (Cohen, 2019)

3.2.6.4 Chosen Methodology

As we have discussed in detail the 3 possible methodology that can be used for this project, scrum methodology is best suited in this case as it is software focussed, fast paced, and allowed moving up and down a phase efficiently in a cost-effective way.

3.2.7 GANTT Chart

Gantt chart is a tool use for planning and scheduling which help assess how long a project should take, determine the resources needed, and plan the order in which the task need to be completed. I choose to use this tool as it helps for managing the dependencies between tasks and it helps ensure that the schedule is workable, that the right efforts are assigned to each task, and that there are workarounds for potential problems before start. As a Gantt chart is a very useful and provide a visual perspective of the schedule, it is more suitable, therefore chosen for success completion of the project. (Kienapple, 2022)

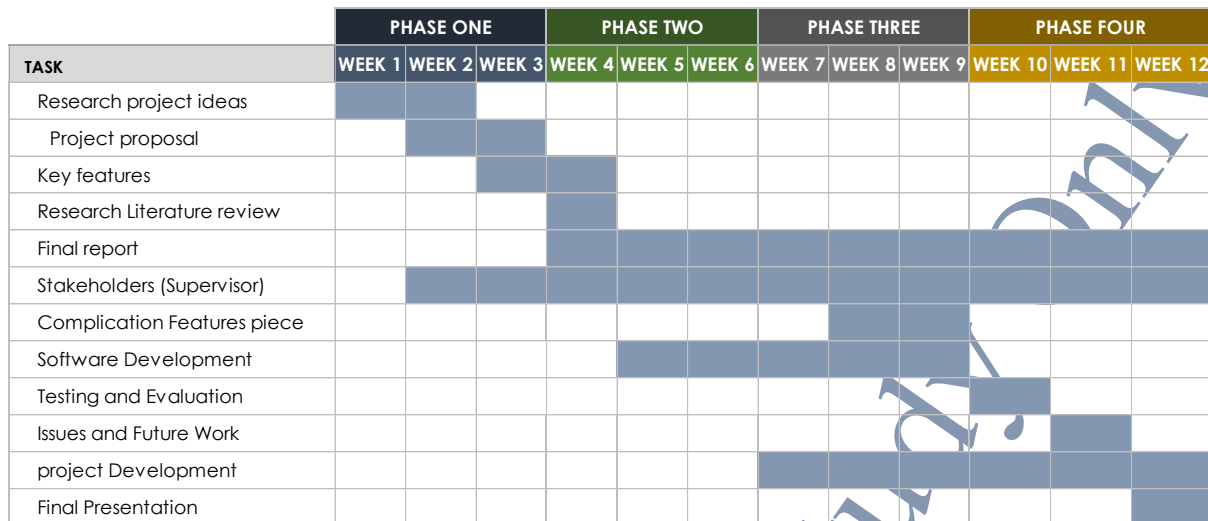


Figure 22:Gantt Chart

3.3. Features of the System

Identified the software design to follow, the next step should be defining what functionalities the voice assistant should cover. By assessing what users would like to be able to achieve and what is possible within the realm of current technology, the author prioritised the functionalities as: chatting services through WhatsApp, mail composition, chatGPT integration, Image generation, event handler, location services, music player service, checking weather, Google searching engine, maintain conversation, tell jokes and so more. Aside from the aforementioned features, VOYA will focus more on features towards helping disabled people such as medication reminder, as according to a research, disabled people receive an average of 40% more prescription medication annually. (Residents, 2000) Voya will use a device's microphone to receive voice requests while the voice output takes place at the speaker. It is a combination of several different technologies: voice recognition, natural processing language, voice analysis and the transition from speech to text.

3.3.1 Task-oriented approach

A task-oriented approach is a process where a developing application can easily collaborate with other application to accomplish a common goal. For example, if a user asks a VOYA to set an alarm for 5AM, it will understand this to be a task request and communicate with the system default Clock application to open and set an alarm for 5AM. Which in return it would communicate with the app to see if anything else was necessary, such as a name for the alarm, then it would communicate this need back to the user. This approach does not require an extensive online database, as it is mainly using the knowledge and already existing process of other installed applications.

VOYA includes the following task-oriented approach:

- Chatting services through WhatsApp,

- Opening Application
- Mail composition,
- music player service,
- Checking weather

3.3.2 Knowledge-oriented approach

A knowledge-oriented approach is the use of analytical data to help users with their tasks. This approach focuses on leveraging online API and internet to help complete tasks. For example, the user asked VOYA “who is Steve job” and this will use the online API available to return relevant results and recommend the highest search result. If someone is searching up a recommendation question on TripAdvisor, this would use a knowledge-oriented approach as it is searching for data instead of working with other apps to complete tasks.

VOYA include the following knowledge-oriented approach:

- Google searching engine,
- Maintain conversation,
- Tell jokes,
- Recommendation,
- ChatGPT

3.4. Software Test Plan

This section will elaborate on the different approach of testing this project requires. It comprises of four types of testing which are, monitoring, manual-based, usability and unit testing. It will also cover different test cases, issues that was encountered during development and the evaluation of the application. As testing is the final phase of this project life cycle, consequently it is the most important phase to ensure the trustworthiness of the application before deployment. The testing phase will undergo different stages as shown below.

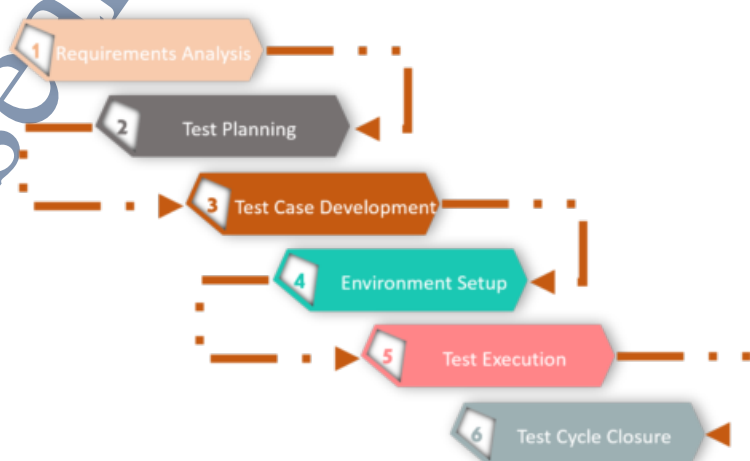


Figure 23: Testing's stages

Testing will consist of different categories in order to have as accurate test cases as possible. In this project, I will undergo the following testing:

3.4.1 CONTINUOUS TESTING (MONITORING)

Once Voya is deployed, the author of this thesis needs to monitor the application at different interval for data consistency and ensure the app works flawlessly. It will be easy to set up and the results will be provided immediately in case the voice-based app stops behaving as expected. There will be a list of monitoring test carried out, such as, a voice test to verify the initial behaviour on how efficient and accurate the voice assistant listening skill is.

3.4.2 MANUAL-BASED TESTING

This testing will consist of a combination of users with Dysarthria (slow in speech), visual impairment users and users with varying accents, dialects and speaking mannerisms to uncover unexpected issues. As new issues are uncovered, it can be reconstructed in a controlled environment to provide a solution.

3.4.3 USABILITY PERFORMANCE TESTING

The main goal is to identify issues with the speech recognition and Natural language processing (NLP) behaviour of the assistant. A comprehensive testing of the interaction model will be conducted by creating a baseline set of results. A training result will be created to compare the training model of the NLP algorithm.

3.4.4 UNIT TESTING

This will ensure the code is working correctly in isolation as each component is being tested for bugs and possibly code improvement. As the development process is completed, the author of this thesis will code a requirement of test cases, into the test script to verify a particular unit's correctness. During test case execution, various frameworks log tests that fail any criterion will be reported in a summary. The goal of this test is to ensure that each and every section meets its design and behaves as expected. Python has a built-in unit testing framework called unittest which will help making the code future proof.

3.4.5 Training Model Evaluation

A trained machine learning model needs to be evaluated in order to determine how well it predicts or makes choices. Evaluation is a mean to evaluate the model's performance on data that it has never encountered before and aids in identifying whether the model is over- or under-fitting the training set.

The evaluation will consist of few methods to understand if the training model is functioning properly. The first method would be to check accuracy which is a measure of the proportion of samples that were properly classified. It is the most widely used evaluation measure, particularly for datasets that are evenly distributed. Precision would be another evaluated which measures the proportion of true positives (positives that were properly classified) among all predicted positives. Recall will calculate the ratio of true positives to all other good results. And the harmonic mean of recall and precision is the F1 number. When we want to combine recall and precision, it is helpful.

As the project rely on intent classification to provide functionality to the user, accuracy is the priority when it comes to the training model. Therefore, the author has decided that it is important to have a metric above 85% when evaluating the accuracy.

3.5. Architecture for voice assistant

The backbone of a Voice Assistant is its architecture.

The architecture of VOYA is split into 3 components as follows:

- i. The Front end
- ii. Middle tier
- iii. Backend

3.5.1 Front end

The software design of VOYA also includes a user interface that enables the user to interact with the software. The front-end is responsible for capturing voice commands and display them into text. The user interface is designed to be intuitive and easy to visualise, allowing users to visualising the process of the features of the software. The software design includes a voice user interface (VUI) that enables the user to understand when the voice assistant is listening or asleep. The VUI is designed to be simple and easy to understand, with clear and concise information displayed to the user. The text data is then sent to the middle tier for processing. The front-end of the voice assistant is critical to the system's success, as it determines the accuracy of voice recognition and ultimately the quality of the assistant's responses.

3.5.2 Middle Tier

The middle tier of the voice assistant is responsible for processing the user's request in the form of vocal data by making use of a microphone and performing the appropriate action. The speech recognition tool from pytsx3 library is used to provide VOYA with the ability to capture spoken words. This component is made up of natural language processing (NLP) algorithms that analyse the text data received from the front-end. These algorithms help to identify the user's intent and extract the relevant information necessary to complete the

task. As a future integration, the middle tier may include machine learning algorithms that allow the system to learn from its interactions with the user and improve its accuracy over time.

3.5.2.1 Speech Recognition

As discussed in the research section, the author has ruled on using speech recognition from Google API services to capture speech from the user through the medium of a microphone.

Speech is converted from physical sound to an electrical signal with a microphone, and then to digital data with an analogue-to-digital converter. Once digitized, several models can be used to transcribe the audio to text.

Speech recognition system rely on an approach known as Hidden Markov Model (HMM). HMM assumption that a speech signal can be reasonably approximated as a stationary process, when viewed on a short enough timescale (ten milliseconds).

In a typical Hidden Markov Model (HMM), the speech signal undergoes division into 10-millisecond fragments. Subsequently, the power spectrum of each fragment, denoting the signal's power against frequency, is transformed into a vector of real numbers, called cepstral coefficients. The dimension of this vector is typically small, ranging from 10 to 32 for more precise systems. The HMM's final output comprises a sequence of these vectors.

To transcribe speech into text, clusters of vectors are compared to one or more phonemes, which are fundamental speech units. This computation necessitates training because the sound of a phoneme differs from speaker to speaker and may even vary across multiple instances of speech by the same speaker. Subsequently, a specific algorithm is utilized to establish the most probable word or words that can generate the provided sequence of phonemes.

The speech recognition uses a recognizer class which is fundamental for the implementation of this project.

3.5.2.2 Intent Classification

Intent classification is a technique used in natural language processing (NLP) to analyses the intentions behind texts and categorizes them such as command and responses. The project will leverage an intent classification class designed to train a neural network model for text classification. The intent classification will consist of few steps to train the data.

As discussed in the research section of this paper a dataset of English vocabulary will be used to have a collection of phrases and its intents. The dataset will be prepared into 'train', 'test' and 'validation' sets. This will be accomplished by merging all the sets and then splitting them using scikit-learn into 'train' and 'validation' sets. This will also create more

training data. In order to process the data further, the data need to be in a numerical form. Therefore, the data need to be tokenized which is breaking the sentences into individuals and convert these individuals into numerical representations by using the Keras Tokenizer libraries.

Lastly, the test set will be used to assess the model's performance on data that it has not been seen during training or validation. It will include a portion of the dataset. The test set precision provides a prediction of the model's performance on brand-new, untested data. As using the validation set for this reason can result in overfitting and overly optimistic performance estimates, it is crucial to use a separate test set to assess the model's performance. The evaluation of the model will be discussed in detail in the evaluation section of this paper.

3.5.3 Backend

The backend of the voice assistant is responsible for executing the requested action and providing a response to the user. This component typically consists of an application programming interface (API) that interacts with various third-party services such as WhatsApp, weather, mail. As each command varies as per user queries, VOYA needs to identify the APIs associated with each query. At this stage there might be parallel processing between TTS and APIs. Once the command has been identified, an output is returned to the user interface.

A higher level of the architecture of VOYA is shown below which demonstrate the stages and level each processes go through.

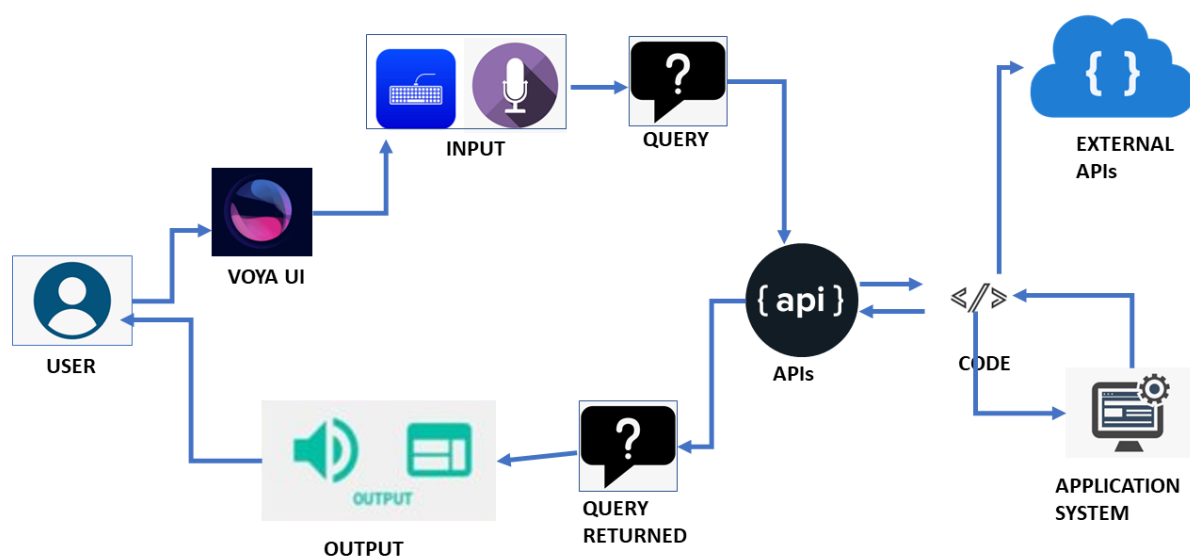


Figure 24: Architecture of VOYA

3.6. Analysis of a command

A command in VOYA has 3 main parts: Wake word, Request, Expression. Below is an example of a command with the mentioned breakdown.

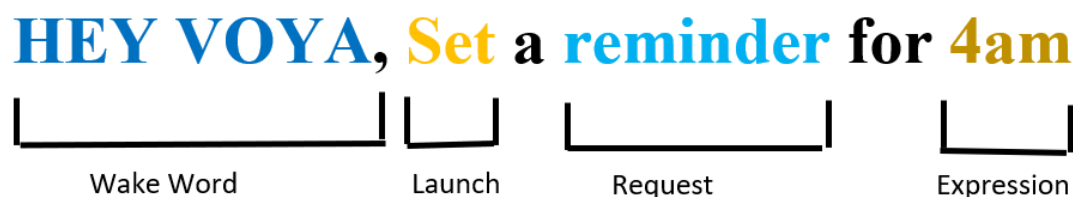


Figure 25: Overview of command for VOYA

3.6.1 Wake word

As the user say, 'Hey VOYA', this will activate the voice assistant to actively listen and be ready to take instructions from users.

3.6.2 Request

The input from the User will have a request combined with an action, command, or question. The request helps VOYA understand what skills to start.

3.6.3 Expression

VOYA identifies the user's intent from the given expression and responds accordingly. So ultimately the expression decides what user wants the voice assistant to perform.

3.7. Conclusion

In conclusion, the software design of VOYA incorporates a variety of technologies to provide an effective voice assistant for users. By following best practices and guidelines from industry leaders like Microsoft, VOYA is designed to provide a user-friendly interface that adapts to different screen sizes and provides clear indicators for listening states. The system includes both task-oriented and knowledge-oriented approaches to cover a wide range of functionalities, including chatting services, mail composition, image generation, event handling, location services, music player services, weather checking, Google searching, and more. Additionally, VOYA prioritizes features that can benefit disabled users, such as medication reminders.

With the system design and first prototype developed, the development of this application's core functionality can take place. The development stages will follow in accordance with the diagrams defined in this section. As this section provides a much better understanding of the opening of this projects, functionalities or approaches can be added through the stages.

To ensure the effectiveness and accuracy of VOYA, various testing methods are employed, including continuous monitoring, manual-based testing with users of varying accents and speech patterns, usability performance testing, and unit testing. These tests are designed to identify and address any issues that may arise, improving the overall functionality and user experience of the system.

The architecture of VOYA is divided into three main components: the front-end, middle tier, and backend. The front-end captures user voice commands and sends them to the middle tier, which processes the request using natural language processing algorithms to identify the user's intent and extract relevant information. The backend then executes the requested action, using APIs to interact with third-party services as necessary.

Overall, VOYA represents a comprehensive and sophisticated voice assistant system that leverages cutting-edge technologies and best practices to provide a powerful and user-friendly experience. The system's architecture and testing methods ensure accuracy and reliability, while the range of features and prioritization of accessibility serve to make the system useful and beneficial for a wide range of users.

4. Software Development

4.1. Introduction

This section will discuss on the development of VOYA, which is built on the Model-View-Controller (MVC) architectural pattern and utilizes a variety of tools and libraries including PyQt5, Keras, and OpenAI. The frontend, middle-tier, and backend, as well as other components and functionalities of VOYA, are covered in depth in this section. To demonstrate the implementation details, code snippets and diagrams are also provided. In order to create a virtual assistant system that offer a seamless and productive user experience, this section ends by emphasizing the significance of using design patterns and putting best practices into practice.

4.2 Software Development

The software development of VOYA will consist of integrating all the component together to create a flow of processes. In order to achieve this, the front end need to be properly designed and tested at each stage of the development. The front end needs to be smoothy integrated with the middle-tier so that it can enable users to interact with the voice assistant.

Integration of the text-to-speech engine to make sure that answers are produced accurately and promptly. The text-to-speech engine will allow the voice assistant to generate voice responses to user input.

Natural language processing (NLP) models will be created and integrated in order to interpret user input and produce responses. For the voice assistant to effectively understand and react to user input, an intent classification model will be built on a large dataset. The APIs functions and author-based function will all be implemented to provide functionality to the assistant on user request.

4.3. Front End

4.3.1 Model-View-Controller

The design and architecture of VOYA are based on the Model-View-Controller (MVC) architectural pattern. This pattern separates the application logic into three main components: the model, the view, and the controller. The model represents the application's data and business logic, the view is responsible for the application's user interface, and the controller handles user input and updates the model and view accordingly.

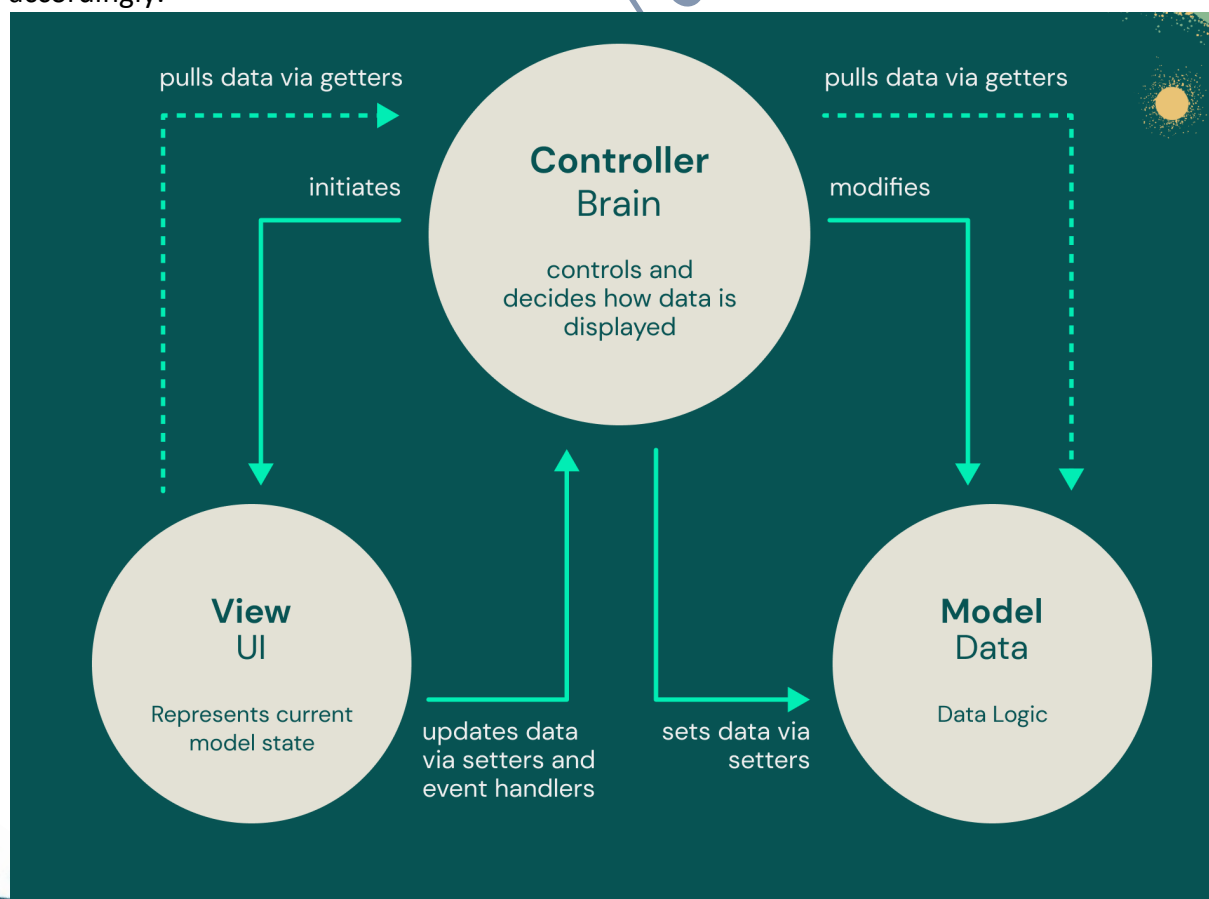


Figure 26: Model View Controller Architecture

4.3.2 Code snippet

The code below uses PyQt5, a popular Python library for creating graphical user interfaces (GUI). PyQt5 provides a wide range of widgets and tools for creating interactive and user-friendly applications. It also includes support for multimedia. In addition, the code uses QMovie, a class in PyQt5 that enables the application to display animated GIFs. The use of animated GIFs adds an element of interactivity and liveliness to the user interface, making the application more engaging and interesting.

```
class Ui_MainWindow(QMainWindow):  
    def __init__(self):  
        super().__init__()   
  
        # Set the background color to white  
        self.setStyleSheet("background-color: black;")  
  
        # Create a QPixmap object with the icon file path  
        icon = QIcon('C:/Users/Angesh/Documents/AI Voice assistant/Voya/images/logo.png')  
        # Set the icon for the main window  
        self.setWindowIcon(icon)  
        # Set the icon for the taskbar application  
        QApplication.setWindowIcon(icon)  
  
        # Create the logo gif widget  
        self.gif_logo = QLabel(self)  
        self.gif_movie_logo = QMovie('C:/Users/Angesh/Documents/AI Voice assistant/Voya/images/face.gif')  
        self.gif_logo.setPixmap(QPixmap(self.gif_movie_logo.currentPixmap()).scaled(100, 100)) # Resize the gif_logo  
        self.gif_logo.setAlignment(Qt.AlignTop | Qt.AlignLeft)  
        self.gif_logo.setMovie(self.gif_movie_logo)  
        self.gif_movie_logo.start()  
  
        self.set_text = QLabel(self)  
        self.set_text.setStyleSheet('font-weight: bold; font-size: 30px;')  
        self.set_text.hide()  
        #self.text_listening.setText("Recognising")
```

Figure 27: Snippet for initializing Voice user Interface

In the code below, the model represents the text files that store the user commands and the AI's responses. The view includes all the graphical elements, such as the logo, labels, and buttons, that make up the user interface. The controller is responsible for handling user input, such as voice commands, and updating the model and view accordingly. The use of the MVC pattern makes the application more modular, scalable, and maintainable, as each component can be developed and tested independently.

```

def hide_gif_logo_and_show_widgets(self):
    self.gif_logo.hide()
    #self.gif_listening.setVisible(True)
    self.text_label.hide()
    self.label.setVisible(True)
    self.label_output.setVisible(True)
    self.set_text.setVisible(True)
    #self.button_widget.setVisible(True)

def display_text(self, text):
    return self.label.setText(text)

# The display_output function uses threading to update the AI output label widget to prevent the GUI from freezing.
# This is because the GUI cannot be updated directly from a separate thread. Instead, the code starts
# a new thread to update the AI output label widget in the background.
def display_output(self, text):
    t = Thread(target=self.label_output.setText(text))
    t.start()

def update_text_from_file(self):
    # Read the contents of the text file
    with open("C:/Users/Angesh/Documents/AI Voice assistant/Voya/text_files/command.txt", "r") as f:
        data = f.read()
        #self.gif_movie.stop()
    # Update the label if the contents have changed
    if data != self.label.text():
        #self.gif_movie.stop()
        self.label.setText("You Said: " + data)

```

Figure 28: Snippet for display output with threading

From the prototype development section, we can see that the voice assistant interface is different compared to the final output. The author realised that the initial interface did not follow the Microsoft guideline discussed in section 3 of this paper for voice assistant in providing proper visual and information of the voice assistant stages. As a result, the author has made changes to the interface to better align with the Microsoft guidelines and provide a better seamless user experience. The figure below shows the welcoming interface of the assistant.

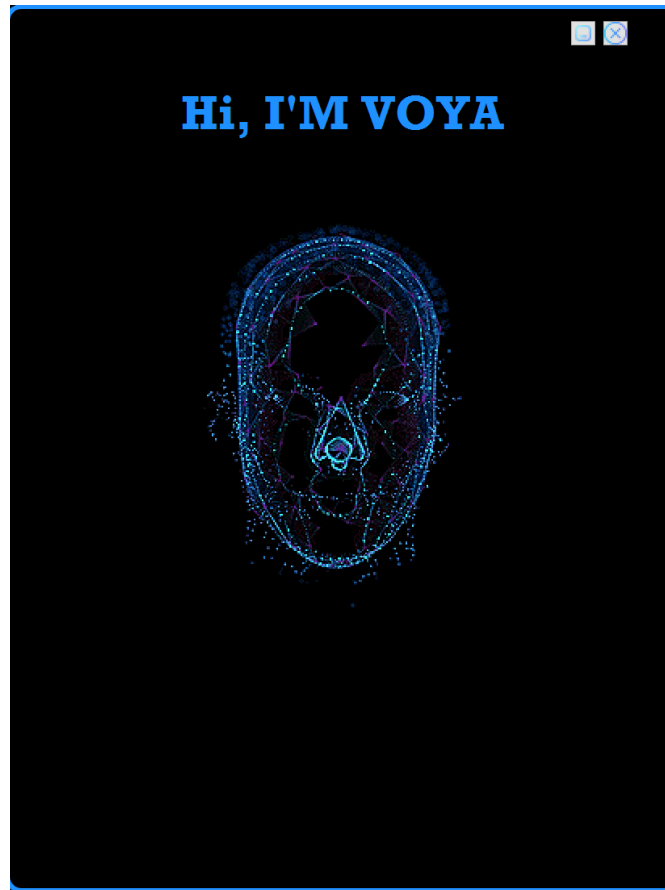


Figure 29: Snippet for Voice user Interface-Welcome

The figure below shows the assistant interface when it waits for the keyword to be awake. This provides a much better user experience as the user knows each stages as they using the assistant.

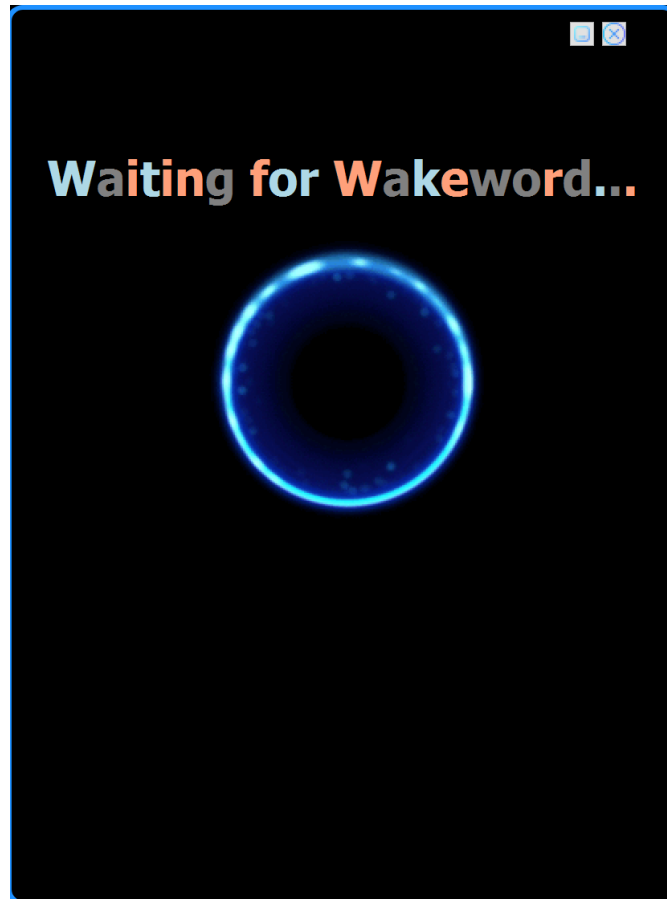


Figure 30: Snippet for Voice user Interface – Keyword detection

4.4 Middle Tier

4.4.1 Speech Recognition

Voya_AI class consist of several methods in the class carry out various tasks. The init() function initializes the object and specifies certain parameters, including the path to the text file used to display the assistant's state and the phrase used to wake up the assistant.

```
class Voya_AI():

    def __init__(self, name):
        print("--- Powering up", name, "---")
        self.name = name
        self.r = sr.Recognizer()
        self.keyword = "assistant"
        self.r.energy_threshold = 8000
        self.speech_detected = False
        self.last_speech_time = time.time()
        self.set_text_path = "C:/Users/Angesh/Documents/AI Voice assistant/Voya/text_files/set_text.txt"
        self.mic = sr.Microphone()
```

Figure 31: Snippet for initializing the Voice assistant object

The listen() method expects request from the user voice command. Before listening for the user's instruction, it waits for the user to pronounce the keyword that will awaken the assistant. The assistant falls back to sleep after ten seconds if the user doesn't speak, waiting for the keyword to be repeated. When the keyword is heard, the wait_for_keyword() method waits for it and alerts the assistant. The speech_detected flag is then set to True, and the last time a speech command was recognized is recorded.

```
def listen(self):
    if not self.speech_detected:
        print("Waiting for wakeword...")
        set_text = open(self.set_text_path, "w")
        set_text.write("Waiting for Wakeword...")
        set_text.close()
        self.wait_for_keyword()
    while True:
        # If keyword detected, process commands
        if self.speech_detected:
            print("Listening...")
            set_text = open(self.set_text_path, "w")
            set_text.write("Listening...")
            set_text.close()
            command = self.voice_input()
            return command
```

Figure 32: Snippet of the listen() method

```
def wait_for_keyword(self):
    self.speech_detected = False
    self.last_speech_time = time.time()

    while True:
        with self.mic as source:
            self.r.adjust_for_ambient_noise(source)
            audio = self.r.listen(source)

        try:
            speech = self.r.recognize_google(audio).lower()
            print(speech)
            if self.keyword in speech: # threshold for a good match
                print("Keyword detected")
                set_text = open(self.set_text_path, "w")
                set_text.write("Keyword detected")
                set_text.close()
                self.speech_detected = True
                self.last_speech_time = time.time()
                break
        except sr.UnknownValueError:
            pass
```

Figure 33: Snippet of the Wake word detection

The user hears the response after the text-to-speech conversion with the `tts()` method. The text is synthesized into speech using the `pyttsx3` package.

```
def tts(self, text):
    speech_to_text = pyttsx3.init('sapi5')
    try:
        speech_to_text.say(text)
        speech_to_text.runAndWait()
        speech_to_text.setProperty('rate', 175)
        self.last_speech_time = time.time() # Reset last_speech_time
        return True
    except:
        t = "I did not understand. Can you repeat again please"
        print(t)
        return False
```

Figure 34: Snippet of the text to speech method

4.4.2 Intent Classification

As discussed in the research section, the algorithm to be used for the training model is LSTM which is a type of recurrent neural network. And the dataset set to be use is CLINIC oos-eval. In order to download the data and the embeddings, the `urllib.request.urlretrieve()` function is used by the `download_files()` function to download the files in a data folder.

```
def download_file(url, file_path):
    try:
        urllib.request.urlretrieve(url, file_path)
        print(f'Downloaded {file_path}')
    except:
        print(f'Error downloading {file_path}')

def download_files(data_url, embedding_url, data_path, embedding_path):
    download_file(data_url, data_path)
    download_file(embedding_url, embedding_path)

# Download data and embeddings
download_files(DATA_URL, EMBEDDING_URL, DATA_PATH, EMBEDDING_PATH)
```

Figure 35: Snippet of dataset download

The `json.loads()` method is used by the `load_data()` function to load the data from the downloaded JSON file. In order to create a singular dataset, the function concatenates the out-of-scope and other intent data with the text and label data it has extracted from the JSON file.

```
# Reading the download JSON file
def load_data(data_path):
    with open(data_path) as file:
        data = json.loads(file.read())

    validation_oos = np.array(data['oos_val'])
    train_oos = np.array(data['oos_train'])
    test_oos = np.array(data['oos_test'])
    validation_others = np.array(data['val'])
    train_others = np.array(data['train'])
    test_others = np.array(data['test'])

    # combining out-of-scope and other intent data
    validation = np.concatenate([validation_oos, validation_others])
    train = np.concatenate([train_oos, train_others])
    test = np.concatenate([test_oos, test_others])

    data = np.concatenate([train, test, validation])
    data = data.T

    return data

data = load_data(DATA_PATH)
text = data[0]
labels = data[1]
```

Figure 36: Snippet of concatenating both intents

Using the Scikit-Learn library's `train_test_split()` method, the dataset is divided into training and testing sets. The `max_len()` function is designed to determine the maximum length of a sequence in the training data in order to establish the `maxlen` parameter for the padding of the sequences later. The pre-trained word embeddings are loaded into a dictionary `embeddings_index` using the `open()` function to open the embedding file and the `split()` function to separate each line of the file into a word and its corresponding embedding vector.

A Keras Tokenizer's `num_words` parameter is initialized with the maximum number of words to keep based on word frequency. The training text is then fed into the tokenizer's `fit_on_texts()` function, which produces a dictionary of word index mappings. The

tokenizer's `texts_to_sequences()` function is then used to generate the training and testing text sequences, and the `pad_sequences()` function is used to stretch the sequences to their utmost length.

The `LabelEncoder` and `OneHotEncoder` classes from Scikit-Learn are used to encode the label data in numerical format that can be use by the machine learning model. The `LabelEncoder` converts the labels into numerical format using the `fit_transform()` method, and the `OneHotEncoder` uses the `fit()` and `transform()` methods to turn these numerical labels into a one-hot encoded format.

```
label_encoder = LabelEncoder()
integer_encoded = label_encoder.fit_transform(classes)

onehot_encoder = OneHotEncoder(sparse=False)
integer_encoded = integer_encoded.reshape(len(integer_encoded), 1)
onehot_encoder.fit(integer_encoded)

train_label_encoded = label_encoder.transform(train_label)
train_label_encoded = train_label_encoded.reshape(len(train_label_encoded), 1)
train_label = onehot_encoder.transform(train_label_encoded)

test_labels_encoded = label_encoder.transform(test_labels)
test_labels_encoded = test_labels_encoded.reshape(len(test_labels_encoded), 1)
test_labels = onehot_encoder.transform(test_labels_encoded)
```

Figure 37: Snippet of fitting the model

The `build_model()` function defines the model architecture, which includes an embedding layer, an LSTM layer, two completely connected layers with dropout, and a final output layer. Keras' `Sequential()` class is used to build a linear stack of levels. The `Embedding()` layer is initialized with the pre-trained word embeddings and made to be non-trainable. The LSTM layer has `return_sequences` set to `False` to output the LSTM's ultimate hidden state. The softmax activation function is used by the output layer to generate a probability distribution over all potential classes. (Pandya, 2020)

The `fit()` technique is used to train the model using the `train_model()` function. `Fit()` takes as input training text sequences and one-hot encoded labels and trains the model with the given hyperparameters. Finally, the trained model, tokenizer, label encoder, and class labels are stored to disk using the `save_objects()` function, which serializes and saves the objects to disk using the `pickle.dump()` function. These saved objects can be used to make predictions on fresh text data later.

```

def train_model(model):
    model_training = model.fit(train_sequences, train_label, epochs = EPOCHS,
                                batch_size = 64, shuffle=True,
                                validation_data=[test_sequences, test_labels])
    return model_training

def save_objects(objs, paths):
    for obj, path in zip(objs, paths):
        with open(path, 'wb') as file:
            pickle.dump(obj, file)

model_build = build_model()
# Training the model
train_model(model_build)

# Save objects
save_objects([labels, tokenizer, label_encoder], [CLASSES_PATH, TOKENIZER_PATH, LABEL_ENCODER_PATH])
model_build.save(MODEL_PATH)

```

Figure 38: Snippet of model training

4.5 Backend

The voice assistant's backend consists of two types of functionalities: API-based functions and author-based function implementations. Based on user inputs, API-based functions are used to retrieve the most recent weather data for example, and author-based function implementations use a datetime function created by the author to retrieve the most recent time and date.

4.5.1 API-Based

4.5.1.1 weather

the fetch_weather function leverages an external Python API to get local weather information. By registering for Visual Crossing's API service, an API key is first obtained from them. Access to the API is authenticated and authorized using this key. The fetch_weather function, which requires a city as an input parameter, therefore the author has created a function that take a city as input parameter. To make sure that the city name can be passed in a URL without risk, it is encoded using the urllib.parse.quote() function.

```

API_KEY=config.VISUAL_CROSSING_API
def fetch_weather(city):
    try:
        encoded_city = urllib.parse.quote(city)
        ResultBytes = urllib.request.urlopen(f"https://weather.visualcrossing.com/VisualCrossingWebServices/rest/services/time1
        # Parse the results as JSON
        jsonData = json.load(ResultBytes)

```

Figure 39: Snippet of retrieving weather data

To send the URL to the Visual Crossing API and get the weather information for the specified city, the author uses `urllib.request.urlopen()`. The JSON response is parsed using the `json.load()` function and then transformed into a Python dictionary.

```
# Parse the results as JSON
jsonData = json.load(ResultBytes)

data = jsonData['days'][0]
```

Figure 40: Snippet of the weather information in the JSON

Using the dictionary's keys, such as temperature, feels like temperature, humidity, and weather description, the pertinent weather information is extracted. To create the final response string, the extracted weather data is formatted into a human-readable string using string interpolation. Exception handling is used to deal with any errors that arise throughout the process, and the proper error message is printed. The `fetch_weather` function returns the final response string.

4.5.1.2 ChatGPT

Modern machine learning models for natural language processing (NLP), such as language translation, text summarization, and question-answering, are accessible through the OpenAI API. Developers can incorporate these strong NLP models into their applications through the API, which enables them to automatically produce excellent text, translate between languages, and carry out a variety of other NLP tasks.

`Assistant_name` and `user_input` are the two parameters that the `chatGPT` function accepts. `User_input` is the text that the user types into the chat interface, whereas `assistant_name` is the name of the virtual assistant or chatbot that the user is interacting with. A variable called `conversation` is set to an empty string inside the function. The entire exchange between the user and the virtual assistant will be recorded in this variable.

```
def chatGPT(assistant_name, user_input):
    conversation = ''
    prompt = user_input + "\n"+assistant_name+": "
    conversation += prompt
```

Figure 41: Snippet of ChatGPT method initialisation

The `user_input` and `assistant_name` strings are combined into the `prompt` variable and separated by a colon. The OpenAI API will use this to create a conversation prompt and produce a response. By using the `openai.Completion.create()` method, which asks OpenAI's API to produce a response to the conversation prompt, the `response` variable is created. The name of the OpenAI model to use and various text generation process parameters, including temperature and `max_tokens`, are specified in the parameters passed to the method.

```
response = openai.Completion.create(  
    model="text-davinci-003",  
    prompt=conversation,  
    temperature=0.7,  
    max_tokens=256,  
    top_p=1,  
    frequency_penalty=0,  
    presence_penalty=0  
)
```

Figure 42: Snippet of ChatGPT model

The generated response text is taken out of the JSON object the API returned and stored in the `response_str` variable. The OpenAI API may have added extra text to the end of the response, this string is then processed to remove it. The `prompt` and `response_str` strings are added to the `conversation` variable, separated by a newline. The `conversation` is then printed in its entirety to the console. The `chatGPT` function returns the `response_str` variable at the end, which can then be used by other parts of the program or shown to the user.

4.5.1.3 Image Generator

The DreamStudio API is a web-based application programming interface (API) that allows developers to access a set of machine learning models that can generate various types of media, such as images, videos, and music, based on user input or prompts.

The `generate_image` function accepts one argument, `text`, which is a string that represents the prompt for the image generation. Using the key parameter, which contains the API key stored in `DREAMSTUDIO_API`, the function creates a new `StabilityInference` client instance from the DreamStudio API. The `verbose` parameter is set to `True`, which enables detailed logging information during image generation. The `generate` method of the `StabilityInference` client is called with the `prompt` parameter set to the `text` argument and the `seed` parameter set to 95456, which is a random seed used to generate the image.

```

DREAMSTUDIO_API = config.DREAMSTUDIO_API

def generate_image(text):
    # Create a new StabilityInference client instance
    stability_api = client.StabilityInference(
        key=DREAMSTUDIO_API,
        verbose=True
    )

    # Generate image(s) with the given text prompt and a random seed
    responses = stability_api.generate(prompt=text, seed=95456)

```

Figure 43: Snippet of the dream studio model

When the generate method returns a generator object, the function iterates over it and looks for any responses that contain an image artifact. If an image artifact is discovered, its binary data is extracted and converted into an image file using the Pillow package's Image module. The show method is then used to display the resulting image. The function prints a warning message indicating that the image cannot be processed and returns without displaying an image if a finish reason of FILTER is discovered in place of an image artifact. The function prints an error message indicating that the image generation failed if no images are produced.

```

# Iterate over the generator and check each response for an image artifact
for response in responses:
    if response is not None and len(response.artifacts) > 0:
        artifact = response.artifacts[0]
        if artifact.type == generation.ARTIFACT_IMAGE:
            image_bytes = io.BytesIO(artifact.binary)
            img = Image.open(image_bytes)
            img.show()
            return
        elif artifact.finish_reason == generation.FILTER:
            print("WARNING: This image cannot be processed. Please try again.")
            return

# If no images were generated, print an error message
print("ERROR: Failed to generate image.")

```

Figure 44: Snippet of generating the image from dream studio

4.5.2 Author based function

The author-based function is functions created by the author to provide a functionality of the assistant.

4.5.2.1 WhatsApp

The `send_whatsapp_message()` function is the function definition that undertake all the whatsapp functionality to send a message by taking three parameters as follows `phone_number`, `message`, and `wait_time`. The `phone_number` is a string that represents the phone number of the receiver, which will be input from the user voice, and it must start with a +. The phone number will be store in a key pair value to easily identify the recipient without worrying about inputting phone numbers. The `message` variable is a string that represents the content of the message that user will input through voice. The `wait_time` is an integer that represents the time (in seconds) to wait before sending the message.

The function launches the WhatsApp web interface in a new tab of the default browser and waits for the user to confirm that the web interface is linked to their phone. It then finds the input box on the web interface and enters the message. Lastly, it sends the message by pressing the "enter" key through a `pyautogui` library.

```
def send_whatsapp_message(phone_number: str, message: str, wait_time: int = 15, close_tab: bool = False) -> None:
    """Send a WhatsApp message using the web interface."""
    if not phone_number.startswith("+"):
        raise ValueError("Phone number must start with a '+'.")

    url = f"https://web.whatsapp.com/send?phone={phone_number}&text={quote(message)}"
    webbrowser.open(url)

    # Wait for user to confirm that WhatsApp web is linked to phone
    while True:
        input_box_position = pyautogui.locateOnScreen("whatsapp_input_box.png", confidence=0.7)
        if input_box_position:
            break
        else:
            time.sleep(5)

    pyautogui.click(pyautogui.center(input_box_position))
    time.sleep(wait_time - 2)
    pyautogui.press("enter")

    if close_tab:
        time.sleep(3)
        pyautogui.hotkey("ctrl", "w")
```

Figure 45: Snippet of WhatsApp text message function

4.5.2.2 Email

The mail function facilitates the mail sending through the assistant by taking four arguments: `sender_email`, which is the email address of the sender, `sender_password`, which is the password for the sender's email account. Both these two variables is configure from the config file and only need to be done once. The `receiver_email`, which is the email address of the recipient, and `msg`, which is the message to be sent. Once the variables are all populated, the `smtplib` module is used to establish a connection to the Gmail SMTP server at `smtp.gmail.com` on port 587. The `ehlo()` method is called to identify the client and the `starttls()` method is called to initiate a secure connection.

```

# IMPORTS
import smtplib

def mail(sender_email, sender_password, receiver_email, msg):
    try:
        mail = smtplib.SMTP('smtp.gmail.com', 587)
        mail.ehlo()
        mail.starttls()
        mail.login(sender_email, sender_password)
        mail.sendmail(sender_email, receiver_email, msg)
        mail.close()
        return True
    except Exception as e:
        print(e)
        return False

```

Figure 46: Snippet of email function

One of the main issues I had with this implementation was the authentication from the mail side despite providing the right email and password. The author researches this issue and understands that security was the concern here and therefore needed to enable two-step verification in the email account. This enables to retrieve password for specific app which is a service offer from google to maintain security.

4.5.2.3 date and time function

The date() function uses the datetime module to retrieve the current date and time, and then formats the date string in the format "day of the week, month day year". For example, if the function is executed on Friday, March 31, 2023, the returned string would be "Friday Mar 31 2023". If an exception occurs while trying to retrieve the date, the function returns False.

```

def date():
    try:
        date = datetime.datetime.now().strftime("%A %b %d %Y")
    except Exception as e:
        print(e)
        date = False
    return date

```

Figure 47: Snippet of the date function

The `time()` function uses the `datetime` module to retrieve the current time, and then formats the time string in the format "hour:minute AM/PM". For example, if the function is executed at 3:45 PM, the returned string would be "03:45 PM". If an exception occurs while trying to retrieve the time, the function returns `False`.

```
def time():
    try:
        time = datetime.datetime.now().strftime("%I:%M %p")
    except Exception as e:
        print(e)
        time = False
    return time
```

Figure 48: Snippet of the time function

4.5.2.4 System Statistics

The `system_stats()` function calls several functions from the `psutil` module to retrieve the current system statistics. Specifically, it calls `cpu_percent()` to retrieve the CPU usage percentage, `sensors_battery()` to retrieve the battery percentage, and `virtual_memory()` to retrieve the memory usage statistics. It then uses the `convert_size()` function to convert the memory usage from bytes to a more readable format. Finally, it formats the data into a string and returns it.

```
def system_stats():
    cpu_percent = psutil.cpu_percent()
    battery_percent = psutil.sensors_battery().percent
    memory_stats = psutil.virtual_memory()
    memory_in_use = psutil._common.bytes2human(memory_stats.used)
    total_memory = psutil._common.bytes2human(memory_stats.total)

    stats_str = (
        f"CPU usage: {cpu_percent}%\n"
        f"Memory usage: {memory_in_use} out of {total_memory}\n"
        f"Battery level: {battery_percent}%"
    )

    return stats_str
```

Figure 49: Snippet of the system stat info to return back

The `convert_size()` function is defined to take a size in bytes as input and convert it into a more human-readable format. It uses the `math` module to calculate the appropriate unit (KB, MB, GB, etc.) and the `round()` function to round the result to two decimal places. This will provide a much more understandable response when the assistant outputs the response. Part of the convert size is sourced on stack overflow. (Sapam, 2013)

```
def convert_size(size_bytes):
    if size_bytes == 0:
        return "0B"
    size_name = ("B", "KB", "MB", "GB", "TB", "PB", "EB", "ZB", "YB")
    i = int(math.floor(math.log(size_bytes, 1024)))
    p = math.pow(1024, i)
    s = round(size_bytes / p, 2)
    print("%s %s" % (s, size_name[i]))
    return "%s %s" % (s, size_name[i])
```

Figure 50: Snippet of converting the stat into human readable format

4.6 Challenges and problems

4.6.1 Wake up word

Wake word detection is a crucial part of the functioning of VOYA, as it is the mechanism by which the assistant is activated and begins listening for user commands. There are several challenges that has arisen when implementing wake word detection:

The wake word detection mechanism needs to be sensitive enough to detect the wake word reliably, but not so sensitive that it is triggered by other words or sounds. This can be a challenging balance to strike, as you want the wake word to be detected reliably but you also want to avoid false positives. Therefore, the solution was to create an isolated function that can have a voice recognizer with the right balance between noise and input capture. After multiple testing through unit test, the recognizer `energy_threshold` was set to 8000. The `energy_threshold` is a parameter in the SpeechRecognition library that sets the minimum sound energy level that must be exceeded in order for the recognizer to consider a segment of audio as speech.

The solution was tested with different situation such as speaking loudly and softly to check the wake-up activation and in a noisy background. However, as the background get noisier, it become harder for the assistant to understand the wake-up word. Alternatively, machine learning algorithms such as neural networks or support vector machines can learn to recognize the unique characteristics of the wake word and distinguish it from other words or sounds.

4.6.2 Training Model

The main focus of training the model was accuracy and it can be hard to achieved high accuracy. One of the main challenges in acquire high accuracy was handling inputs of varying lengths, which may result in input data not having the same form. Because the model assumes that the incoming data will have a fixed shape, this can lead to issues during model training.

Padding is one method for coping with variable-length inputs in the context of intent classification. In order to make sure that all inputs are the same length, this entails adding zeros to the input data. The sequences are padded to a specific length in the supplied code using the Keras `pad_sequences` function. It is essential to remember that padding can also bring about its own problems, such as raising the model's memory requirements and possibly hiding crucial information in the input data. Padding can also result in extremely sparse input data, which can make it less efficient when working with very long sequences.

4.6.3 Voice User Interface

Threading is used to update the AI output label widget to prevent the GUI from freezing. This is because GUI cannot be updated directly from a separate thread. Instead, the code starts a new thread to update the AI output label widget in the background. The `display_output` function starts a new thread and passes the `setText` method of the `label_output` widget as the target function to be executed in the new thread. This allows the GUI to continue running and responding to user input while the target function is executed in the background, providing a better user experience.

4.6.4 GDPR Law for voice data

One of the main risk factors to voice assistant is the law toward privacy of voice data. "Voice print collections are no different from other types of personal information that need to be protected," said Larry Ponemon, chairman and founder of the Ponemon Institute.

"Voice print collections are no different from other types of personal information that needs to be protected." (THE RISKS OF VOICE TECHNOLOGY, 2022)

As per Article 9 GDPR covers and restricts the "processing of personal data revealing racial or ethnic origin, political opinions, religious or philosophical beliefs, or trade union membership. (Parewyck, 2021) An encryption method such as multifactor authentication should be used so that the data cannot be easily accessed by cyber criminals. The European Union's General Data Privacy Regulations (GDPR) impose strict rules regarding the protection of EU citizens' personal data which go into effect in May 2018 and, including protection against unlawful processing, data violations or accidental loss of data. Alternatively, if dealing with data in the future, VOYA can leverage the voice-enabled systems to store the data directly on a device or locally, rather than in the cloud.

In order to follow the GDPR law, VOYA is not collecting any data as there are not server connection involved to upload data for analysis.

4.6.5 Accents

People pronounce different words differently. Strong or unfamiliar accents can be really difficult to understand even for humans. Accents are very similar to another language, complete with their own vocabulary, syntax, and phonetic rules. Language differences have generated complexities to create accent-agnostic speech recognition systems nearly as challenging as offering distinct languages. In some situations, creating a completely different model for the accent as if it was another language is the best option to ensure it is trained for all patterns that may arise. Different speech recognition providers perform better or worse with different accents, so it's important to find a solution that works best for VOYA. (Stephens, 2021)

It is hard for speech recognition to be perfect. The best approach to improve the accuracy of speech recognition is to apply machine learning technology that can utilize contextual understanding. (Whitehead, 2021) However this was a challenge to the project that was not implemented yet due to have a limited amount of time to produce the agreed requirement of this project.

4.7 Conclusion

In conclusion, the Model-View-Controller (MVC) pattern, which divides the application logic into three major components: the model, the view, and the controller, is the basis for the design and architecture of VOYA. The front end contains support for multimedia files like animated GIFs and uses the PyQt5 library to build a user-friendly graphical user interface. The CLINIC oos-eval dataset and the LSTM algorithm are used for speech identification and intent classification in the middle tier. The back end contains both author-based and API-based features, such as sending emails and WhatsApp messages, retrieving weather information, and giving system statistics.

To overcome the encountered challenges, it was necessary to balance the wake word detection mechanism's sensitivity. High accuracy was also needed when training the model with variable-length inputs, and a clear and unambiguous voice user interface had to be created. Testing and the implementation of suitable solutions, such as padding for variable-length inputs and threading for GUI updates, were used to handle these issues.

5. Evaluation

5.1 Introduction

The accuracy, speed, usability, flexibility, and security of VOYA were evaluated in detail in this section. The report also describes the different testing techniques used, such as unit testing, questionnaires, and tester interviews. It also includes a summary of the testers' comments. The report offers both encouraging comments and helpful criticism, as well as possible fixes for the problems mentioned. Overall, the report offers insightful analysis of VOYA's advantages and disadvantages as well as suggestions for enhancing its functionality and design to better serve its users.

5.2 Software Evaluation

5.2.1 Evaluation Criteria

A number of factors need to be taken into account when evaluating VOYA. These standards consist of:

- Accuracy

VOYA's accuracy refers to its capacity to comprehend and interpret the user's commands and inquiries. A voice assistant that is highly accurate should be able to recognize and respond to a variety of commands and questions.

- Speed

The processing time and response time for user commands and queries are referred to as VOYA's speed. A quick voice assistant should be able to answer user questions right away.

- Usability

The usability of VOYA refers to how easy it is for users to interact with the software. A highly usable How simple it is for users to interact with the software is referred to as VOYA's usability. A voice assistant with high usability should have a user-friendly interface that is straightforward and intuitive.

- Adaptability

The ability of VOYA to adjust to various user preferences and environments is referred to as flexibility. It should be possible for a voice assistant to adjust to various environments, languages, and accents.

- Security

VOYA's security refers to how well it can safeguard user information and privacy. Strong security features that safeguard user information and thwart unauthorized access should be present in a voice assistant that is highly secure.

5.2.2 Plan for evaluation

The table below provide some evaluation criteria for the online questionnaire and interview that will be conducted in Questionnaires and Interviews Evaluation section below.

Objective	Control	Evaluation
Online Survey for accessibility, understanding, input analysis, Design, Information	Running an online survey which involves getting feedback from a group of selected users compromising of 20 participants. The survey will be rated under these headers: accessibility, understanding, input analysis, Design, Information. The survey will be taking only few minutes and have simple questions to complete. The survey will have 10 points rating system.	If the survey has an average score of 8 points, then the voice assistant is considered fully functional. If the survey has a score under 8 points, then the website needs to be revised and improved.
Monitoring	Monitor the features of the assistant.	A scale will be devised to provide accuracy on each feature on a scale of 1 to 10.
Interviews	Conducting interview with stakeholders with a question-based approach.	Feedback will be given.

5.2.3 CONTINUOUS TESTING (MONITORING)

The author of this thesis has monitored the application at different interval for data consistency and ensure the app works flawlessly and conduct continuous testing each week of the life cycle. Based on the continuous testing findings, VOYA has an 80% accuracy rate in voice module listening is concerning. This suggests that there may be issues with the app's speech recognition capabilities, which could be due to noisy background environments that make it difficult for the app's speech recognizer to understand clear input. The author may need to explore ways to improve the app's speech recognition capabilities, such as incorporating noise cancellation technology or train a model to understand noisy background to eliminate the noise.

Additionally, the discovery of bugs in the listening methods during continuous testing is not uncommon and is to be expected during the software development process. It is important for the author to document these bugs and work on fixing them as soon as possible to ensure that the app functions properly and provides a positive user experience.

The table below provide some test scenario conducted by the author and its outcomes while conducting continuous testing.

No	Test Scenario	Test Steps	Expected Results	Results
1	Open Application	Launch any application from the desktop with voice	The app should open within 5 seconds.	The app opened within 5 seconds.
2	Command	Take user voice input.	The voice input should be greater than 90% accurate.	Due to background noise 90% accuracy is hard to achieve
3	Skills	Allocating the skill as per command.	Is the correct skill being assigned to user request.	Correct skill as per request
4	Application Integration	Access an application through VOYA	Proper access to application should be given to perform read/write task	Working
5	Accents	Give a command to voya in a different accent	Voya should be able to interpret the command	Future phase
6	Language	Command given in a different language other than English	Other language is not implemented yet.	Future phase
7	Noise	Implement a noisy background while giving a command.	Command interpretation must be over 50% to consider success.	Success

5.2.4 UNIT TESTING

Unit Testing will ensure the code is working correctly in isolation as each component is being tested for bugs and possibly code improvement. As the development process is completed, the author of this thesis has conducted few units testing. During test case execution, various frameworks log tests has failed and required immediate attention for mediation. The goal of this test is to ensures that each and every section meets its design and behaves as expected. Python have a build-in unit testing framework called unittest which will help making the code future proof.

Unit Test	Description	State
test_response_type	This test the code for the accepted response type such as string, bool, or list	Pass
test_response_content	This test if the function is able to return the correct accuracy to the command	Pass
test_long_response	This test case ensures that	Fail

	the assistant's responses are concise and do not exceed a certain length	
test_train_model	This test case ensures that the classifier's model training function is working correctly and that the model is being trained successfully and accurately.	Pass

Due to the time constraint and the implementation of different component of the assistant was challenging, the author was not able to conduct the manual-based testing which would consist of a combination of users with Dysarthria (slow in speech), visual impairment users and users with varying accents, dialects and speaking mannerisms to uncover unexpected issues.

5.3 Specific evaluation

5.3.1 Training Model Evaluation

In order to evaluate the training model, the author uses sklearn library to produce measurements from the model. The test_pred variable includes the predicted probabilities for each class for each test instance, whereas test_pred_classes contain the predicted class labels for each test instance based on the highest chance. The names of the true classes for each test instance are enumerated in test_true_classes. The scikit-learn accuracy_score function is used to compute the accuracy score, which measures the proportion of instances that were correctly predicted. Additionally, the precision_score, recall_score, and f1_score functions are used to calculate accuracy, recall, and F1-score.

```
# Evaluate the model
test_pred = model.predict(test_sequences)
test_pred_classes = np.argmax(test_pred, axis=1)
test_true_classes = np.argmax(test_labels, axis=1)

# Calculate accuracy score
accuracy = accuracy_score(test_true_classes, test_pred_classes)
print("Accuracy: {:.2f}%".format(accuracy*100))
precision = precision_score(test_true_classes, test_pred_classes, average='macro')
print('Precision:', precision)
recall = recall_score(test_true_classes, test_pred_classes, average='macro')
print('Recall:', recall)
f1 = f1_score(test_true_classes, test_pred_classes, average='macro')
print('F1-score:', f1)
```

Figure 51: Snippet for evaluating the training model

According to the evaluation's findings below, the model had a test set accuracy of 86.88%. This indicates that 86.88% of the test set's instances had their class label accurately predicted by the model. The model has a reasonable balance of precision and recall, according to the precision, recall, and F1-score results, with an F1-score of 0.877. Recall measures the percentage of real positives among all real positives, whereas precision measures the percentage of real positives among all true positive forecasts. The harmonic means of recall and accuracy, known as the F1-score, offers a combined measurement of the two.

Accuracy: 86.88%
Precision: 0.8790478539320864
Recall: 0.886204267002007
F1-score: 0.8778928989806635

Figure 52: Test set Accuracy metrics

The figure 53 below show the metrics for the train set. It can be concluded from these two findings that there are not much different in the accuracy of the test set compared to the training set.

Accuracy: 93.80%
Precision: 0.9408431948251265
Recall: 0.9444476051370774
F1-score: 0.9399889092021892

Figure 53: Train set Accuracy metrics

With these findings the author was satisfied as the training model was not overfitting and could generalize with new unseen data such as the test set. Therefore, did not carry any further complex evaluation at this stage.

5.4 Questionnaires and Interviews Evaluation

5.4.1 Questionnaire

The Questionnaire follows a user-centric design approach in which the author is evaluating the usability of the system. In addition, the author follows the evaluation criteria listed in software evaluation section to measures the accuracy, speed, flexibility and security. The questionnaire is an online web-based form accessible at [Questionnaire Form](#) which was created on jotform.com refer to appendix 1.

The tester selected for the evaluation are user that were not subjected in the development phase of VOYA and therefore has not been exposed to the functionalities. This will avoid the author compromising the usability testing by inadvertently providing additional knowledge to the tester. At time of testing, the testers' knowledge about the assistant was that it was a voice assistant that would facilitate the day-to-day task of the desktop user. All testers understood the definition of a voice assistant.

A prototype of VOYA was then installed on their machine with instruction on how to launch it. A link to the feedback form was sent to their email individually and requires detailed feedback at each step of issues encountered.

The author's approach of not leading the testers during the evaluation of VOYA was important to ensure unbiased feedback and to get a full understanding of how users interact with the voice assistant.

5.4.2 Interviews

The author considers it helpful to conduct interviews with each tester which can provide valuable insights into their experiences with the voice assistant, including how they use it, what features they find most useful, and any issues or challenges they may have encountered. The 15-minute interview can provide a more in-depth understanding of the tester's experiences with the voice assistant and can allow the author to gather more detailed feedback and suggestions for improvements.

During the interview, the author has asked the following questions to better understand the tester's interactions with the voice assistant:

- i. What about the voice assistant did you find most enjoyable?
- ii. Which specific attributes or capabilities of the voice assistant do you find most useful?
- iii. Did you run into any problems or difficulties while using the voice assistant? If so, could you give us some details?
- iv. How confident are you in the precision and dependability of the voice assistant?
- v. Since using the voice assistant, have you noticed any changes in your behaviour or routine?
- vi. How could the voice assistant be improved to better serve your needs, in your opinion?
- vii. How at ease are you with using the voice assistant in public?
- viii. Have you previously used any other voice assistants? How does this one compare, if at all?

By conducting interviews with each tester, the author can gain a more comprehensive understanding of the voice assistant's strengths and weaknesses and can use this feedback to improve the voice assistant's design and functionality.

5.4.3 Positive feedback

All testers successfully received the feedback form from the chatbot. The results the author found to be positive were:

- 90% reported that the voice assistant was accurate.
- 100% reported that VOYA was easy to use.
- 70% reported VOYA help save time in conducting task.
- 50% reported using most of the functionalities.
- 40% reported using VOYA frequently.

From the above findings we can identity that the testers were generally able to utilise the voice assistant and did appreciate the overall user experience.

5.4.4 Constructive criticisms

It is important to consider the feedback provided by the testers and evaluate potential solutions to address any issues identified. Implementing solutions to address these issues can help improve the chatbot's usability, user satisfaction, and overall effectiveness. The testers provided very helpful criticisms of the Voice assistant, which has been listed below.

Criticism	Issue	Solution
Could benefit from a user-by-user Personalized voice assistant	This is more a future state functionality than an issue.	Leverage Machine Learning to train the system to adapt to user-by-user basis
The time of listening is too short, and I was unable to make long command sentences	This issue seemed to happen more often when the tester was conducting long searches.	Increase the time of active listening. - Resolved
The "I did not understand" response from the assistant was not informative enough to user	This issue seemed to happen when a tester was querying something that assistant could not allocate a function to.	Create a try and catch method to handle this error and provide a response. - Resolved

5.5 Conclusion

In conclusion, the assessment of VOYA has given useful information about the software's accuracy, speed, usability, flexibility, and security. Testers gave the software high marks for accuracy and usability, but they also noted areas for improvement, such as customization options, added functionality, and stronger speech recognition capabilities. The report also

emphasizes the value of ongoing testing and monitoring as well as the application of various testing techniques, such as unit testing, surveys, and interviews, to assess the functionality and usability of the software. The author can work to enhance the look and feel of VOYA and give users a better overall experience by taking into account the helpful criticism offered by the testers. Overall, the analysis has produced insightful findings that can be put to good use.

6. Conclusion and Future Plan

6.1 Introduction

This section will conclude this thesis and discuss on what was the important steps. Additionally, the future plan consists of several key area such as a data integration and word prediction.

6.2 Conclusion

The author of this thesis concludes that voice assistant has potential success in the disabled community. For people with disabilities who might find it challenging to complete particular chores on their own, voice assistants can offer a degree of independence and autonomy. Additionally, they can be modified to satisfy the particular requirements of people with disabilities, such as by giving verbal input or utilizing specialized vocabulary.

Python has proven to be best suited for the development of voice assistant due to the vast libraries' framework available. Microsoft framework has provided the necessity to produce the best user interface for the voice assistant. The author expect that the framework will only improve overtime and provide even more user-friendly interface guidelines.

Even though the implementation of the intent classification was the complex task of this project, it turns out to be the most necessary implementation to provide an intelligent voice assistant. Overtime, the author is expecting to provide even a large amount of data to the model to feed on and constantly improve the accuracy of the model.

A key is that is important to this project's success was the software design, as it defined the framework, architecture, approach, and functionalities which are all criteria for successful implementation. Early identification of software requirement can help with the project planning and development phase.

6.3 Future Plan

6.3.1 Database integration

A database integration can be beneficial to authenticate user identity. The database will store the user id and associate all command and output to that id. This will allow to mine that data and train the NLP model to self-learn. And also, the voice assistant will implement an extra level of security which will associate each id to a specific user.

Features can be stored in a database approach for semantic query. This will allow cross platform usage as information will be available through database.

6.3.2 Machine learning for predictive word

Voice assistant can have difficulties understanding command from the users. The approach it takes is to ask the user to repeat the command and if still the command is no clear enough, the voice assistant enter a loop of asking the user to repeat. In order to eliminate this issue, the author have look into implementing an auto completion query. an auto completion query is a process for the voice assistant to predict the missing word in the command to produce an acceptable final output as a query. For the purpose of implementing this solution, the author will research on all the models that are available for prediction such as classification, clustering or forecast model.

Bibliography

- [1] Amos, D. (2022, March 30). *Python GUI Programming With Tkinter*. Retrieved from realpython: <https://realpython.com/python-gui-tkinter/>
- [2] Anna Esposito, M. F.-Z. (2021). *Progresses in Artificial Intelligence and Neural Systems*. Singapore : Springer Singapore.
- [3] Arteaga, C. (2022, August 6). *Understand BLOOM, the Largest Open-Access AI, and Run It on Your Local Computer*. Retrieved from towardsdatascience: <https://towardsdatascience.com/run-bloom-the-largest-open-access-ai-model-on-your-desktop-computer-f48e1e2a9a32>
- [4] Baranwal, S. (2020, February 17). *Understanding BERT*. Retrieved from Towards AI: <https://pub.towardsai.net/understanding-bert-b69ce7ad03c1>
- [5] Borycka, S. I. (2021, March 05). *What are the advantages of voice assistants?* Retrieved from linkedin: https://www.linkedin.com/pulse/what-advantages-voice-assistants-sylwia-ilona-borycka/?trk=public_profile_article_view
- [6] Bowen, R. (2020, January 11). *What is the Value of PMBOK?* Retrieved from Bright Hub Project Management: <https://www.brighthubpm.com/certification/19380-why-you-should-be-familiar-with-pmbok/>
- [7] Brown, B. (2018, October 18). *Amazon's Alexa app gets new interface to simplify use, control more devices*. Retrieved from digitaltrends: <https://www.digitaltrends.com/home/amazon-updates-alexa-app/>
- [8] Budiu, R. (2018, July 22). *Intelligent Assistants Have Poor Usability: A User Study of Alexa, Google Assistant, and Siri*. Retrieved from Nielsen Norman Group: <https://www.nngroup.com/articles/intelligent-assistant-usability/>
- [9] Cohen, E. (2019, July 14). *The Definitive Guide to Project Management Methodologies*. Retrieved from Workamajig: <https://www.workamajig.com/blog/project-management-methodologies>
- [10] *Design assistant experiences for Windows 10*. (2022, April 22). Retrieved from microsoft: <https://learn.microsoft.com/en-us/azure/cognitive-services/speech-service/windows-voice-assistants-best-practices>
- [11] *DFI Infographic IRELAND*. (2019, March 04). Retrieved from DISABILITY FEDERATION OF IRELAND: <https://www.disability-federation.ie/publications/dfi-infographic-ireland/>
- [12] *Disability and Health Promotion*. (2022, November 11). Retrieved from Centers for Disease Control and Prevention: <https://www.cdc.gov/ncbddd/disabilityandhealth/infographic-disability-impacts-all.html>

- [13] Eckroth, J. (2018). *Python Artificial Intelligence Projects for Beginners*. Mumbai: Packt Publishing Ltd.
- [14] Flannagan, D. (2005). *JAVA in a Nutshell*. In *A Desktop quick reference* (pp. 1-17). Tokyo: O'Reiley Media INC.
- [15] Guarnieri, R. A. (2006). *Solar radiation forecast using artificial neural networks in south Brazil*. Brazil: Researchgate.
- [16] Gupta, A. (2023, January 30). *An Introduction to Scikit-Learn: Machine Learning in Python*. Retrieved from simplilearn: <https://www.simplilearn.com/tutorials/python-tutorial/scikit-learn>
- [17] Hochreiter, S. (1997). LONG SHORT-TERM MEMORY. In S. Hochreiter, *LONG SHORT-TERM MEMORY* (pp. 6-21). 80290 Munchen, Germany: Neural Computation MIT Press.
- [18] Hughes, A. (2023, March 16). *ChatGPT: Everything you need to know about OpenAI's GPT-4 tool*. Retrieved from sciencefocus: <https://www.sciencefocus.com/future-technology/gpt-3/>
- [19] Ina. (2022, March 15). *The History Of Chatbots – From ELIZA to ChatGPT*. Retrieved from onlim: <https://onlim.com/en/the-history-of-chatbots/>
- [20] Irene Lopatovska, K. R. (2018). Talk to me: Exploring user interactions with the Amazon Alexa. *SAGE Journals Home*, 1-3.
- [21] Jacob Devlin, M.-W. C. (2018). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*.
- [22] Kellner, M. (2020, July 07). *Digital Voice Assistant for Data Retrieval in Reporting*. Retrieved from detecon: <https://www.detecon.com/en/journal/digital-voice-assistant-data-retrieval-reporting>
- [23] KIAN LONG TAN, C. P. (2022). Fundamentals of Recurrent Neural Network (RNN) and Long Short-Term Memory (LSTM) network. Malaysia: IEEE Access.
- [24] Kienapple. (2022, Novemeber 11). *11 Gantt Chart Examples and Templates For Effective Project Management*. Retrieved from VENNgage: <https://venngage.com/blog/gantt-chart-example/#3>
- [25] Knight, S. (2019). *NLP at Work: The Difference that Makes the Difference*. London: Hachette UK.
- [26] KRISHNA, S. (2022, July 12). *BigScience's open-source LLM BLOOM has landed*. Retrieved from analyticsindiamag: <https://analyticsindiamag.com/bigsciences-open-source-llm-bloom-has-landed/>
- [27] Kuchi, R. A. (2016, January 05). *All About Virtual Voice Assistants, NLPs, and Building Your Own Voice Assistant*. Retrieved from studentsxstudents:

<https://studentsxstudents.com/all-about-virtual-voice-assistants-natural-language-processing-and-speech-recognition-ae04f854bc59>

- [28] Lutkevich, B. (2023, March 11). *BERT language model*. Retrieved from techtarget: <https://www.techtarget.com/searchenterpriseai/definition/BERT-language-model>
- [29] Lynch, A. (2023, February 05). *Behind-the-Scenes: The Tech Behind ChatGPT(super friendly ver.)*. Retrieved from Medium: <https://medium.com/@anixlynch/behind-the-scenes-the-tech-behind-chatgpt-super-friendly-ver-2a214c4dc284>
- [30] Melissa Moyer, I. C. (2020). *Social Inclusion and the Impact of Voiceitt on Identity and Talk in Persons with*. Barcelona: European Commission, Horizon.
- [31] MUTCHLER, A. (2017, July 14). *Voice Assistant Timeline: A Short History of the Voice Revolution*. Retrieved from voicebot.ai : <https://voicebot.ai/2017/07/14/timeline-voice-assistants-short-history-voice-revolution/>
- [32] Norvig, P. (1992). *Paradigms of Artificial Intelligence Programming: Case Studies in Common Lisp*. California: Morgan Kaufmann.
- [33] O'Boyle, B. (2021, November 2). *What is Siri and how does Siri work?* Retrieved from pocket-lint: <https://www.pocket-lint.com/apps/news/apple/112346-what-is-siri-apple-s-personal-voice-assistant-explained>
- [34] Pandya, H. (2020, October 20). *Creating Your own Intent Classifier*. Retrieved from medium: <https://medium.com/analytics-vidhya/creating-your-own-intent-classifier-b86e000a4926>
- [35] Parewyck, J. (2021, June 16). *When is voice (a special category of) personal data under GDPR?* Retrieved from linkedin: <https://www.linkedin.com/pulse/when-voice-special-category-personal-data-under-gdpr-janvier-parewyck/>
- [36] PMI. (2015). *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. fifth edition.
- [37] Predicting User's Intent from text using machine learning methods. (2018). In A. Katsalis. THESSALONIKI – GREECE.
- [38] Ramos, L. P. (2022, August 29). *Python and PyQt: Building a GUI Desktop Calculator*. Retrieved from realpython: <https://cc.com/python-pyqt-gui-calculator/>
- [39] Ravikiran. (2023, February 13). *A Guide to Speech Recognition in Python: Everything You Should Know*. Retrieved from simplilearn: <https://www.simplilearn.com/tutorials/python-tutorial/speech-recognition-in-python>

- [40] Residents, M. B. (2000, July 31). *DISABILITY, MEDICARE, AND PRESCRIPTION DRUGS*. Retrieved from The White House: <https://clintonwhitehouse4.archives.gov/WH/EOP/nec/html/ParkMedicareReport000730.html>
- [41] Ruby, M. (2023, January 30). *How ChatGPT Works: The Model Behind The Bot*. Retrieved from towardsdatascience: <https://towardsdatascience.com/how-chatgpt-works-the-models-behind-the-bot-1ce5fca96286>
- [42] Sapam, J. (2013, Febraury 11). *Better way to convert file sizes in Python [closed]*. Retrieved from stackoverflow: <https://stackoverflow.com/questions/5194057/better-way-to-convert-file-sizes-in-python>
- [43] Sidiq, S. (2023, January 03). *An Essential Python Text-to-Speech Tutorial Using the pytsx3 Library*. Retrieved from hackernoon: <https://hackernoon.com/an-essential-python-text-to-speech-tutorial-using-the-pyttsx3-library>
- [44] *sirikit*. (2022, Novemeber 11). Retrieved from developer.apple.com: <https://developer.apple.com/documentation/sirikit?language=objc>
- [45] Snips Voice Platform: an embedded Spoken Language Understanding system for private-by-design voice interfaces. (2018). In A. S. Alice Coucke, *Snips Voice Platform: an embedded Spoken Language Understanding system for private-by-design voice interfaces* (pp. 2-16). Paris, France.
- [46] *snips_built_in_intents*. (2023, March 22). Retrieved from Hugging Face : https://huggingface.co/datasets/snips_built_in_intents
- [47] Stefan Larson, A. M. (2019). An Evaluation Dataset for Intent Classification and Out-of-Scope Prediction. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)* (pp. 1311–1316). Hong Kong, China: Association for Computational Linguistics.
- [48] Stephens, K. (2021, September 02). *Why Voice Assistants Need to Understand Accents*. Retrieved from SoundHound: <https://www.soundhound.com/voice-ai-blog/why-voice-assistants-need-to-understand-accents/#:~:text=Each%20of%20these%20accents%20has,bad%20reviews%2C%20or%20failed%20purchases.>
- [49] *THE RISKS OF VOICE TECHNOLOGY*. (2022, November 11). Retrieved from Association for Federal Enterprise Risk Management: https://www.aferm.org/erm_feed/the-risks-of-voice-technology/
- [50] Thomson, i. (2011, October 24). *Father of Lisp and AI John McCarthy has died*. Retrieved from The Register: https://www.theregister.com/2011/10/24/father_lisp_ai_john_mccarthy_dies/

- [51] Voiceitt learns your clients' unique speech patterns. (2023, March 11). Retrieved from voiceitt: <https://voiceitt.com/for-organizations/>
- [52] Wangari, A. (2023, January 20). *WPF vs WinForms*. Retrieved from section: <https://www.section.io/engineering-education/wpf-vs-winforms/>
- [53] *What Is Scrum Methodology?* (2020, January 11). Retrieved from Collabnet versionone: <https://resources.collab.net/agile-101/what-is-scrum>
- [54] Whitehead, K. (2021, January 25). *3 Common Voice Assistant Errors, and How to Avoid Them*. Retrieved from PolyAI: <https://poly.ai/how-to-avoid-building-an-annoying-voice-assistant/>
- [55] Wu, J. (2017). *Introduction to Convolutional Neural Networks*. Nanjing: LAMDA group.
- [56] Zhou, V. (2022, September 16). *Machine Learning for Beginners: An Introduction to Neural Networks*. Retrieved from victorzhou: <https://victorzhou.com/blog/intro-to-neural-networks/>

Customer Satisfaction Survey

Your feedback is valuable to us! Help us improve V.O.Y.A by sharing your thoughts in this customer survey.

Personal Information

Your privacy is important to us. The information provided will only be used to contact you regarding VOYA.

Email *

Survey

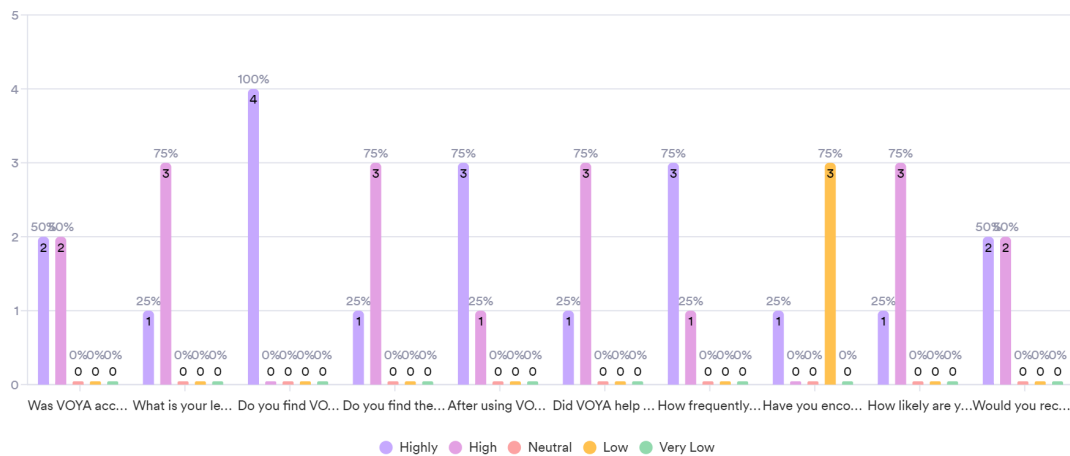
Please evaluate to your best *

	Highly	High	Neutral	Low	Very Low
Was VOYA accurate?	☐	☐	☐	☐	☐
What is your level of satisfaction after using VOYA?	☐	☐	☐	☐	☐
Do you find VOYA easy to use?	☐	☐	☐	☐	☐
Do you find the VOYA's responses on your desktop helpful and informative?	☐	☐	☐	☐	☐
After using VOYA, do you feel more comfortable using voice commands to accomplish your tasks?	☐	☐	☐	☐	☐
Did VOYA help you save time?	☐	☐	☐	☐	☐
How frequently do you use VOYA on your desktop?	☐	☐	☐	☐	☐
Have you encountered any technical issues or errors while using VOYA on your desktop?	☐	☐	☐	☐	☐
How likely are you to use VOYA again to assist you?	☐	☐	☐	☐	☐
Would you recommend VOYA to others who use desktops?	☐	☐	☐	☐	☐

Comments

Do you have any feedback or improvement suggestions

VOYA Questionnaire



Appendix 2 : Analytics of questionnaire's responses